



AMERICAN UNIVERSITY OF RAS AL KHAIMAH

Faculty of Engineering

Computer Science and Engineering Department

CASE STUDY

Linux's Debian 13.4 - "Trixie"

A Comprehensive Operating System Analysis

Course Title	Operating Systems Fundamentals
Course Code	CSCI 312
Semester	Spring 2026
Submission Date	April 30, 2026

#	Student Name	Student ID
1	Aleksei Kovalev	2023005964
2	Fares Masarani	2023005970
3	Omar Al Nuaimi	2023006311
4	Ali Alhaeri	2023006084
5	Yaman Khaldoun Alrashed	2023006308
6	Miadah Shalaby	2023005979
7	Mohamed AbouSaada	2023006306

Executive Summary

This case study presents a comprehensive technical and ethical analysis of Debian GNU/Linux 13.4 "Trixie," conducted as part of the Operating Systems Fundamentals course (CSCI 312) at the American University of Ras Al Khaimah. The report examines Debian's internal operating system mechanisms, evaluates its design principles, and documents hands-on experimentation through a virtualized environment.

The technical analysis covers five core OS areas. In process and thread management, the study examines the unified `task_struct` representation used by the Linux kernel, the NPTL threading model, and Debian 13's significant transition from the Completely Fair Scheduler (CFS) to the Earliest Eligible Virtual Deadline First (EEVDF) algorithm, which improves latency and fairness for modern workloads. The section on synchronization primitives details the full hierarchy of locking mechanisms available in the kernel — mutexes, semaphores, spinlocks, read-write locks, and Read-Copy-Update (RCU) — alongside the kernel's lockdep framework for deadlock prevention. Memory management is analyzed in depth, covering virtual address spaces, four-level page tables, the Buddy System allocator, the SLUB kernel object allocator, demand paging, Copy-on-Write, and LRU-based page replacement. The kernel architecture evaluation assesses the Linux kernel's monolithic-yet-modular structure, the kernel/user mode separation enforced through system calls, and the implications of this design for performance and maintainability. The security model analysis addresses user and group-based access control, pluggable authentication mechanisms, the principle of least privilege, and kernel-level enforcement through Linux Security Modules (LSM).

The legal and ethical section examines Debian through the lens of intellectual property (GPL licensing and the separation of free and proprietary components), user privacy, system

security and patch governance, accountability through the Debian Social Contract and Constitution, and responsible use of computing resources.

The implementation section documents the practical deployment of Debian 13.4 on a VirtualBox virtual machine, including installation, configuration of monitoring tools (top, htop, strace), process and thread analysis, system call tracing, and stress testing using stress and stress-ng. Observations from these experiments directly corroborated the theoretical concepts discussed in the analytical sections.

Overall, Debian 13.4 "Trixie" demonstrates a mature, well-structured operating system design that balances performance, stability, flexibility, and security — making it a compelling subject for a comprehensive operating systems case study.

Table of Contents

Executive Summary	2
1. Introduction.....	7
2. Operating System Analysis.....	9
2.1 Process and Thread Management, and Scheduling.....	9
2.1.0 Introduction.....	9
2.1.1 Internal Representation and Lifecycle of Processes and Threads.....	9
2.1.2 Synchronization Mechanisms and Computational Scheduling	11
2.1.3 Resource Control and the System Hierarchy	14
2.1.4 Multi-Core Load Balancing and Architecture Support	16
2.1.5 Monitoring and Diagnostic Frameworks	17
2.1.6 Security Infrastructure for Process Isolation.....	18
2.2 Synchronization Primitives	19
2.2.0 Introduction.....	19
2.2.1 Mutex Locks.....	20
2.2.2 Semaphores	20
2.2.3 Spinlocks.....	21
2.2.4 Read-Write Locks.....	21
2.2.5 Read-Copy-Update (RCU).....	21
2.2.6 Classical Synchronization Problems Debian Context.....	22
2.2.7 The Four Necessary Conditions.....	22
2.2.8 Deadlock Prevention via Lock Ordering	22
2.2.9 The lockdep Tool.....	23
2.2.10 Livelock	23
2.2.11 Deadlock Detection and Recovery (User Space).....	23
2.3 Memory Management.....	24
2.3.0. Introduction	24
2.3.1 Virtual Memory and Process Address Spaces	24
2.3.2 Paging: The Foundation of Memory Isolation	24
2.3.3 Four-Level Page Table (x86-64).....	25
2.3.4 Page Faults.....	25
2.3.5 Physical Memory Allocation: The Buddy System.....	25
2.3.6 How the Buddy System Works.....	25
2.3.7 Memory Zones.....	26
2.3.8 Limitations and Mitigation.....	26
2.3.9. Kernel Object Allocation: The SLUB Allocator	26

2.3.10 Slab Allocator Principles	26
2.3.11 The SLUB Implementation in Debian 13	27
2.3.12 Demand Paging and Page Replacement	27
2.3.13 Copy-on-Write (COW).....	27
2.3.14 Page Replacement LRU Approximation.....	27
2.3.15 Transparent Huge Pages and Memory Compaction.....	28
2.3.16 Monitoring Memory Management on Debian 13	28
2.4 Kernel Architecture Evaluation	29
2.4.0 Introduction.....	29
2.4.1 The Linux Kernel as the Core of Debian.....	30
2.4.2 Monolithic Structure with Modular Design	31
2.4.3 Kernel Mode, User Mode, and the System-Call Interface.....	32
2.4.4 Major Internal Kernel Areas Relevant to Debian	33
2.4.5 Kernel Modules and Debian Maintainability	34
2.4.6 Evaluation of Debian Kernel Architecture.....	35
2.5 Security Model.....	36
2.5.0 Introduction.....	36
2.5.1 User and Group Based Access Control	37
2.5.2 Authentication and Pluggable Mechanisms	38
2.5.3 Least Privilege, Compartmentalization, and Defence in Depth	39
2.5.4 Linux Security Modules and Kernel-Level Security Enforcement.....	39
2.5.5 Protection Rings and Privileged Separation.....	40
2.5.6 Evaluation and Concluding Thoughts of Debian's OS Security Model.....	41
3. Legal and Ethical Judgment in OS.....	42
3.1 Intellectual Property	42
3.2 User Privacy	43
3.3 System Security	43
3.4 Accountability and Governance.....	44
3.5 Responsible Use of Computing Resources	45
4. Implementation.....	46
4.1 Virtual Machine Setup	46
4.2 Operating System Installation	48
4.3 Installation of System Monitoring and Tracing Tools	50
4.4 Process and Thread Analysis	53
4.5 System Call Tracing.....	60
4.6 Stress Testing and Performance Evaluation.....	63

4.7 Summary.....	67
5. Conclusion	68
6. References.....	70
Section 2.1 References	70
Sections 2.2 and 2.3 References	74
Sections 2.4 and 2.5 References	75
Ethical Considerations.....	75
Appendix A: System Information.....	77
Appendix B: Key Commands Reference.....	77

1. Introduction

Debian GNU/Linux is one of the oldest and most influential Linux distributions in existence, first released in 1993 and maintained ever since by a global community of volunteer developers. Unlike commercially driven distributions, Debian is governed by a set of founding principles — the Debian Social Contract and the Debian Free Software Guidelines — which prioritize user freedom, transparency, and long-term stability above all else. These values have made Debian not only a widely deployed operating system in its own right, but also the foundational base upon which many other distributions, including Ubuntu, Kali Linux, and Raspberry Pi OS, are built.

The subject of this case study is Debian 13.4 "Trixie," the latest stable point release in the Debian 13 series, running on the Linux 6.12 LTS kernel for the x86-64 architecture. This release is particularly significant from an operating systems perspective because it incorporates several major kernel-level changes — most notably the replacement of the long-standing Completely Fair Scheduler (CFS) with the newer Earliest Eligible Virtual Deadline First (EEVDF) algorithm, full official support for the RISC-V (riscv64) architecture, and continued maturation of the cgroup v2 resource management framework under systemd.

The goal of this case study is to analyze Debian 13.4 as a complete operating system, examining how its internal mechanisms implement the core concepts taught in Operating Systems Fundamentals. The report is structured around five major technical domains: process and thread management, synchronization primitives and deadlock management, memory management, kernel architecture, and the security model. Each domain is analyzed with reference to established OS theory — primarily Silberschatz, Galvin, and Gagne's *Operating System Concepts* (10th ed.) — and grounded in the specific implementations found in Debian's Linux kernel and userspace tools.

Beyond the technical analysis, the report also addresses the legal and ethical dimensions of operating system design and use. Topics include Debian's intellectual property framework under the GNU General Public License, its approach to user privacy and data minimization, security governance through the Debian Security Team, accountability structures defined in the Debian Constitution, and the ethical implications of responsible computing resource use.

The analysis is complemented by a practical implementation section, in which Debian 13.4 was installed on a VirtualBox virtual machine and subjected to hands-on experimentation. Using tools such as htop, strace, and stress-ng, the team directly observed process lifecycle management, system call behavior, memory allocation under pressure, and scheduler response to high-load conditions. These experiments serve to bridge the gap between theoretical concepts and real operating system behavior.

Together, the analytical and practical components of this report provide a thorough evaluation of Debian 13.4 as both a technically sophisticated and ethically grounded operating system.

2. Operating System Analysis

2.1 Process and Thread Management, and Scheduling

2.1.0 Introduction

The Debian GNU/Linux distribution occupies a unique position within the open-source ecosystem, serving as the foundational bedrock for countless derivative systems while maintaining a rigorous commitment to stability, security, and architectural neutrality. As the project moves toward its thirteenth major release, codenamed “Trixie,” the internal mechanisms governing process management and computational scheduling have undergone a transformative evolution. This analysis examines the sophisticated interplay between the Linux kernel’s task representation, the Native POSIX Threads Library (NPTL) implementation within the GNU C Library (glibc), and the paradigm-shifting transition from the Completely Fair Scheduler (CFS) to the Earliest Eligible Virtual Deadline First (EEVDF) algorithm. The synchronization of these components ensures that Debian can seamlessly manage workloads ranging from high-concurrency server environments to latency-sensitive desktop applications across a diverse array of hardware architectures, including x86-64, ARM64, and the newly officially supported RISC-V.

2.1.1 Internal Representation and Lifecycle of Processes and Threads

In the Debian architectural model, the distinction between a process and a thread is implemented with a high degree of fluidity, a characteristic of the Linux kernel’s unified task management system. Every schedulable entity is represented by a process descriptor, specifically the `task_struct` structure, which contains the totality of information required for the kernel to manage, schedule, and secure a task. This structure is not merely a record of execution state; it is a complex nexus of pointers to memory management structures, file descriptors, signal handlers, and security credentials.

The kernel maintains these process descriptors in a circular doubly linked list, known as the task list. Each `task_struct` is allocated within the kernel's memory space and serves as the primary handle for the scheduler and other kernel subsystems. Within Debian, the management of these structures is critical for maintaining system responsiveness. When a new task is created—whether through the `fork()` or `clone()` system calls—the kernel must allocate and initialize a new descriptor, ensuring that it inherits the appropriate attributes from its parent while establishing its own unique identity through a Process Identifier (PID).

The PID namespace provides a layer of isolation that is fundamental to Debian's support for containerization and virtualization. Through PID namespaces, a group of processes can be granted a private view of the PID hierarchy, where the first process in the namespace is assigned PID 1, mimicking the role of the system's global init process. This allows for the nesting of process trees, which is a core requirement for tools like LXC and `systemd-nspawn` that are widely utilized within the Debian community.

Credential Synchronization and Signal Handling

One of the most complex aspects of thread management in a POSIX-compliant system like Debian is ensuring that all threads within a process share the same user and group identifiers (UIDs and GIDs), even though the Linux kernel treats credentials as a per-thread attribute. To achieve this, NPTL implements a sophisticated wrapping mechanism for system calls like `setuid()` and `setgid()`. When one thread changes its credentials, NPTL uses a real-time signal—specifically signal number 33—to notify all other threads in the process to perform the same change.

Similarly, signal handling is managed on a per-process basis to satisfy POSIX requirements. Signal 32 is reserved by NPTL for thread cancellation and POSIX timers. These signals are hidden from user applications to prevent interference with the library's internal operations. The kernel facilitates this by providing the `tgkill()` system call, which allows signals

to be targeted at specific threads within a specific process, ensuring that the library can maintain internal consistency without disrupting the application's logic.

Table 1

Comparison of LinuxThreads and NPTL Threading Models

Architectural Feature	Legacy Model (LinuxThreads)	Modern Debian Model (NPTL)
Management Logic	User-space Manager Thread	Kernel-level Direct Management
Threading Model	1:1 (Limited)	1:1 (Full SMP/NUMA Support)
Credential Sync	Inconsistent across threads	Synchronized via Signal 33
Signal Handling	Per-thread focus	Per-process focus (POSIX compliant)
PID Consistency	Different PIDs for threads	Unified PID via getpid()
Scalability	Bottlenecked by manager	Highly Scalable across thousands of cores

2.1.2 Synchronization Mechanisms and Computational Scheduling

To minimize the overhead of thread synchronization, Debian relies on Fast User-space Mutexes (futexes). A futex allows threads to attempt to acquire a lock in user space using atomic instructions. If the lock is uncontended, the operation completes without a kernel transition, providing extremely low latency. Only when contention occurs does the thread perform a system call to be put to sleep by the kernel. This mechanism is integral to the performance of the `pthread_mutex_t` and `pthread_cond_t` primitives used by almost all multi-threaded software in the Debian repository.

The interaction between glibc and the kernel is further refined through the use of Thread Local Storage (TLS), which provides each thread with its own private data area. In Debian, this is typically implemented using the `%fs` or `%gs` segment registers on x86-64, allowing for high-speed access to thread-specific variables without the need for complex indexing or locking.

The release of Debian 13 “Trixie” marks a historic shift in the distribution’s scheduling philosophy. For nearly two decades, the Completely Fair Scheduler (CFS) served as the default for general-purpose workloads. However, as the diversity of workloads increased—encompassing everything from high-throughput batch processing to ultra-low-latency interactive applications—the limitations of CFS became apparent. Debian has now transitioned to the Earliest Eligible Virtual Deadline First (EEVDF) scheduler, a model that provides superior theoretical guarantees and practical responsiveness.

The Theoretical Basis of Fair Scheduling

Both CFS and EEVDF are rooted in the concept of the “ideal multi-tasking processor.” This theoretical construct assumes a CPU that can execute N tasks simultaneously, each receiving exactly $1/N$ of the processing power. In such a system, every task would always be “perfectly fair,” receiving its allocated share of CPU time at every instant. Since real hardware can only execute one task at a time per core, the kernel must use “virtual time” to track how much service each task has received relative to this ideal.

The core metric in this model is the virtual runtime (ρ), which represents the actual runtime of a task normalized by its weight (priority). CFS aimed to minimize the disparity between the virtual runtimes of all tasks, always selecting the task with the smallest ρ to run next. EEVDF extends this by introducing the concept of “lag” (L), defined as the difference between the service a task should have received in an ideal system and the service it actually received.

The lag L_i of a task i at time t can be expressed as the difference between its ideal and actual service, weighted by w_i (the task’s weight) and $\rho(t)$ (the average virtual runtime of all active tasks). A positive lag indicates that the task is “owed” CPU time, while a negative lag indicates it has exceeded its fair share.

The EEVDF Selection Algorithm

Unlike CFS, which only used ρ to make decisions, EEVDF utilizes two criteria: eligibility and deadlines

Eligibility Filter: A task is considered eligible to run only if its lag is zero or greater. This prevents “greedy” tasks that have already over-consumed their share from being selected, even if they have a low virtual deadline.

Deadline Selection: Among all eligible tasks, EEVDF selects the one with the earliest virtual deadline. The deadline is calculated based on the task’s virtual runtime and its allocated time slice. By picking the earliest deadline, the scheduler naturally prioritizes tasks that have requested smaller time slices, which are typically latency-sensitive applications. This decoupling of throughput (controlled by weight) and latency (controlled by the time slice) is the primary advantage of EEVDF over CFS.

Performance Implications and Server Optimization

The transition to EEVDF in Debian 13 has resulted in measurable performance improvements for many desktop and multimedia workloads, with some benchmarks showing gains of up to 13%. However, the shift also necessitates new optimization strategies for server-class workloads. For instance, high-load SQL databases like PostgreSQL or MariaDB may experience performance degradation under the default EEVDF settings due to “fairness” mechanisms that were originally optimized for interactive desktop use.

In Debian 13, administrators are advised to tune the scheduler for such environments by modifying specific kernel parameters. The “Automatic Grouping” feature, which allocates CPU time collectively to all processes started from a single terminal or service, can be disabled to ensure that individual database worker threads are not throttled as a group.

Table 2

EEVDF Scheduler Tuning Parameters for High-Load Server Environments

Optimization Parameter	Default State	Recommendation for High-Load Servers
kernel.sched_autogroup_enabled	1 (Enabled)	0 (Disabled) to prevent group throttling
RUN_TO_PARITY	Enabled	Disabled (NO_RUN_TO_PARITY) to reduce latency spikes
PLACE_LAG	Enabled	Disabled (NO_PLACE_LAG) for immediate wakeup priority
sched_min_granularity_ns	750,000	May be increased to reduce context-switch frequency
sched_wakeup_granularity_ns	1,000,000	Adjusted based on task sensitivity to preemption

The sched_ext Framework and eBPF Schedulers

One of the most innovative features included in the Debian 13 kernel is the `sched_ext` framework. This technology allows the implementation of custom scheduling policies using the Extended Berkeley Packet Filter (eBPF). For the first time, Debian administrators can load and modify the scheduler's logic dynamically without the need for a kernel recompile or system reboot.

The `sched_ext` interface provides hooks for key scheduling operations, such as `select_cpu()`, `enqueue()`, and `dispatch()`. This enables the development of specialized schedulers for specific use cases. For example, the `scx_qmap` scheduler provides a multi-level FIFO implementation with five levels of priority, while `scx_central` demonstrates a central decision-making model that can be useful for certain types of high-performance computing (HPC) workloads.

2.1.3 Resource Control and the System Hierarchy

Process management in Debian is intrinsically tied to `systemd`, which manages the execution environment through the use of Linux Control Groups (cgroups) version 2. Unlike the legacy cgroup v1, which used separate hierarchies for different resources, cgroup v2 implements a unified hierarchy that allows for more consistent and efficient resource accounting.

Slices, Scopes, and Service Encapsulation

systemd organizes the process tree into three primary unit types to facilitate resource management. Slices are container units that define a branch in the cgroup tree; they do not contain processes themselves but serve to group other units. Debian defines `system.slice` for background services and `user.slice` for user-initiated processes. Services encapsulate processes managed directly by systemd; configuration directives in the service file allow for precise control over the CPU, memory, and I/O resources available to the service. Scopes are used for processes that are started by an external process and then registered with systemd, which is common for desktop sessions and container runtimes.

Fine-Grained Resource Allocation

Through the systemd interface, Debian provides administrators with powerful tools for apportioning system capacity. The `CPUWeight=` directive allows for proportional scheduling; a service with a weight of 200 will receive twice the CPU time of a service with a weight of 100 when both are competing for cycles. This weight maps directly to the `cpu.weight` file in the cgroup v2 filesystem.

For workloads that require hard limits, the `CPUQuota=` directive can be employed. This sets a maximum percentage of CPU time that a unit can consume within a given period (defaulting to 100ms). This is implemented via the kernel's bandwidth controller, ensuring that even if a process enters an infinite loop, it cannot starve the rest of the system.

Memory and I/O Management Integration

The unified hierarchy of cgroup v2 in Debian 13 allows for a more holistic approach to resource management. For instance, the kernel can now track “Pressure Stall Information” (PSI), which provides real-time metrics on how much time a cgroup is delayed due to resource contention. This is utilized by `systemd-oomd`, a userspace out-of-memory killer that can

preemptively terminate processes that are causing extreme memory pressure, thereby preserving the overall stability of the Debian system.

Table 3

systemd Resource Control Parameters and cgroup v2 Interfaces

Resource Controller	systemd Parameter	cgroup v2 Interface	Purpose
CPU (Proportional)	CPUWeight=	cpu.weight	Fair-share distribution
CPU (Limit)	CPUQuota=	cpu.max	Hard cap on utilization
Memory (Limit)	MemoryMax=	memory.max	Hard cap on memory usage
Memory (Protection)	MemoryLow=	memory.low	Guarantees minimum RAM
I/O (Weight)	IOWeight=	io.weight	Prioritizes disk access
Tasks (Limit)	TasksMax=	pids.max	Prevents fork bombs

2.1.4 Multi-Core Load Balancing and Architecture Support

The efficiency of process management in Debian is heavily dependent on the kernel's ability to distribute tasks across multiple cores while respecting the physical topology of the system. This is particularly critical in modern NUMA (Non-Uniform Memory Access) systems where the cost of accessing memory can vary significantly depending on which CPU is executing the task.

Scheduling Domains and Load Balancing

The Linux kernel used by Debian models the system's topology as a hierarchy of "scheduling domains." Each domain represents a set of CPUs that share certain resources, such as a last-level cache (LLC) or a local memory controller. The scheduler tick periodically triggers the `run_rebalance_domains()` function, which attempts to move tasks from overloaded CPUs to idle ones.

In Debian, this load balancer is highly sensitive to the cost of migration. Moving a task between cores that share a cache is inexpensive, while moving a task between NUMA nodes (different physical sockets) can result in a significant performance penalty due to remote memory access. The kernel-level Memory-aware Load Balancing (kMLB) mechanism, sometimes integrated into high-performance Debian kernels, further refines this by tracking the memory footprint of tasks and preferring to migrate tasks whose memory is already local to the target node.

Architecture-Specific Considerations

Debian's support for a wide array of architectures necessitates architecture-specific tuning of the task management subsystems. On x86-64, Debian utilizes advanced features like Intel Hybrid Cluster Scheduling and Call Depth Tracking to improve performance and mitigate speculative execution vulnerabilities. For ARM64, Debian 13 introduces hardening against Return-Oriented Programming (ROP) and Jump-Oriented Programming (JOP) attacks using Pointer Authentication (PAC) and Branch Target Identification (BTI). The 64-bit RISC-V architecture (`riscv64`) has achieved official support in Debian 13, bringing all process management and scheduling features to this burgeoning hardware ecosystem. Finally, the i386 architecture has been transitioned to a secondary role, primarily intended to support legacy 32-bit code on 64-bit systems; there is no longer an official Debian kernel or installer for pure i386 hardware in the Trixie release.

2.1.5 Monitoring and Diagnostic Frameworks

To effectively manage processes in a Debian environment, administrators rely on a tiered ecosystem of monitoring utilities that provide visibility into the scheduler's behavior and the system's resource consumption.

Real-Time Interactive Monitors: `top` and `htop`

The `top` command remains the standard for immediate process introspection, providing a live view of PIDs, CPU and memory usage, and task states. Debian users frequently prefer `htop`, which offers a more intuitive interface with support for mouse interaction, scrolling, and a clear visual representation of per-core CPU utilization.

Deep Diagnostics and Historical Analysis: `atop` and `nmon`

For complex performance troubleshooting, `atop` is unparalleled. Unlike the standard monitors, `atop` can record binary snapshots of system state at regular intervals. This allows an administrator to “replay” the state of the system from a previous point in time, identifying exactly which process caused a spike in disk I/O or network traffic hours after the event occurred. `nmon`, originally developed for AIX and now a staple of Debian administration, provides a high-density, single-screen dashboard covering CPU, memory, network, and disk performance.

Advanced Tracing and Kernel Debugging

For developers working at the kernel-user interface, Debian provides a suite of advanced tracing tools. `ftrace` allows for the monitoring of scheduler events like `sched_switch` and `sched_wakeup`, providing microsecond-level visibility into how long tasks are waiting in the runqueue. The `sched_ext` framework further enhances this by providing built-in tracers that can observe the interaction between eBPF-based policies and the core scheduler.

2.1.6 Security Infrastructure for Process Isolation

Process management in Debian is not solely a matter of performance; it is fundamentally intertwined with the system’s security architecture. The kernel must ensure that processes remain isolated from one another and that a compromise in one task does not lead to a system-wide failure.

Vulnerability Mitigation and Patch Management

The Debian security team maintains a rigorous process for backporting fixes for critical vulnerabilities. Recent examples include patches for CVE-2025-21700, a use-after-free vulnerability in the network scheduler that could lead to privilege escalation, and CVE-2025-

21647, a buffer overflow in the CAKE traffic shaper. These updates are typically delivered through the `stable-security` and `lts-security` repositories, ensuring that even users on older releases like Bullseye or Bookworm remain protected.

Hardening the Process Environment

Beyond reactive patching, Debian 13 includes proactive hardening features designed to restrict the attack surface of processes. To mitigate the risk of eBPF-based attacks, such as those targeting the verifier (CVE-2026-1203), administrators can disable unprivileged eBPF access via the `kernel.unprivileged_bpf_disabled` sysctl. While namespaces provide isolation, they also introduce complexity; Debian has implemented fixes for race conditions in mount namespaces (CVE-2026-2891) that could otherwise allow for unauthorized file access. Finally, the arm64 architecture in Debian 13 benefits from PAC and BTI, which provide hardware-level protection against the corruption of function pointers and return addresses, a common technique in modern exploit chains.

2.2 Synchronization Primitives

2.2.0 Introduction

Modern operating systems must coordinate concurrent processes and threads that share resources such as memory, files, and hardware devices. Without careful coordination, two or more processes may enter a race condition a situation in which the system's outcome depends on the non-deterministic ordering of operations leading to corrupted data or inconsistent system state. Debian 13 "Trixie", built on the Linux 6.x LTS kernel, implements a rich set of synchronization primitives and deadlock-management strategies directly inherited from, and extending, the POSIX standard.

As Silberschatz, Galvin, and Gagne define it in *Operating System Concepts* (10th ed., Chapter 6): "A race condition occurs when several processes access and manipulate the same data concurrently and the outcome of the execution depends on the particular order in which the access takes place." Preventing such conditions is the central purpose of synchronization.

The Critical-Section Problem

At the heart of synchronization theory lies the critical-section problem. A critical section is a segment of code in which a process accesses shared data. According to Silberschatz et al. (Chapter 6), any satisfactory solution must satisfy three conditions:

- **Mutual Exclusion:** At most one process may be inside its critical section at any given time.
- **Progress:** If no process is in its critical section and some processes wish to enter, selection of which process enters next cannot be postponed indefinitely.
- **Bounded Waiting:** A bound must exist on the number of times other processes are allowed to enter their critical sections after a process has made a request to enter its own critical section.

Debian's Linux kernel enforces all three conditions through hardware primitives (atomic instructions) and higher-level software constructs described below.

2.2.1 Mutex Locks

The most fundamental synchronization mechanism in Debian's kernel is the mutex (mutual exclusion) lock. Silberschatz et al. (Chapter 6) describe a mutex lock as a tool that "protects critical sections and thus prevents race conditions." A process must call `acquire()` before entering a critical section and `release()` upon exiting it.

In Debian 13's kernel (Linux 6.x), the kernel-space mutex is implemented via the struct mutex data structure with the following key fields:

```
struct mutex {
    atomic_long_t  owner;          // Current owner of the mutex
    spinlock_t     wait_lock;     // Protects the waiting queue
    struct list_head wait_list;    // Queue of waiting tasks
};
```

The kernel uses a three-stage lock-acquisition strategy. In the fast path, if the mutex is unheld, an atomic compare-and-exchange (`cmpxchg`) instruction claims it instantly. If the owning task is still running on another CPU, the kernel enters optimistic spinning before suspending the requesting task a mechanism that dramatically reduces context-switch overhead in multi-core systems common on Debian server deployments.

At the user-space level, Debian's GNU C Library (glibc) exposes POSIX mutexes through the `pthread_mutex_t` type. A typical usage pattern is:

```
pthread_mutex_lock(&mutex);
/* critical section */
pthread_mutex_unlock(&mutex);
```

Mutex ownership is strictly enforced: only the thread that acquired the lock may release it, which is a key distinction from semaphores discussed next.

2.2.2 Semaphores

Silberschatz et al. (Chapter 6) describe a semaphore as "a more robust tool that can behave like a mutex lock but can also provide more sophisticated ways for processes to synchronize their activities." A semaphore *S* is an integer variable accessed only through two atomic operations: `wait()` (historically called *P* or down) and `signal()` (historically called *V* or up).

Debian 13 provides two varieties of semaphores:

- **Binary Semaphore (value 0 or 1):** Functions identically to a mutex lock. Used for mutual exclusion where ownership tracking is unnecessary.
- **Counting Semaphore:** The integer value can range over an unrestricted domain and is used to control access to a resource pool with a finite number of instances. For example, a counting semaphore initialized to *N* allows up to *N* processes to simultaneously access a resource.

In the Linux kernel used by Debian 13, the kernel semaphore API includes:

```
void sema_init(struct semaphore *sem, int val);
void down(struct semaphore *sem);      // wait() -- decrements
void up(struct semaphore *sem);        // signal() -- increments
```

A thread blocked on `down()` is placed into either `TASK_UNINTERRUPTIBLE` or `TASK_INTERRUPTIBLE` state depending on whether it should be woken by a signal. This design integrates neatly with Debian's process scheduler.

2.2.3 Spinlocks

When a lock is expected to be held for a very short time and sleeping is not permitted (such as in interrupt handlers), Debian 13's kernel uses spinlocks. A spinlock causes a thread to busy-wait repeatedly testing the lock until it becomes available. While this wastes CPU cycles, it avoids the overhead of a context switch, making it the correct choice in interrupt context where going to sleep is forbidden. The key rule in Debian's kernel: only a spinlock can be used in interrupt context; only a semaphore or mutex can be held while a task sleeps.

2.2.4 Read-Write Locks

Debian 13 provides read-write semaphores and locks (`rwlock_t`) which permit multiple simultaneous readers but only one writer. This is a performance optimization for data structures that are read frequently but written rarely for example, the kernel's routing tables. The Linux kernel implements rwlocks with fair queuing to prevent writer starvation.

2.2.5 Read-Copy-Update (RCU)

A uniquely powerful mechanism in the Linux kernel used extensively in Debian 13 is Read-Copy-Update (RCU). RCU allows reads to proceed without any locking whatsoever. When a writer needs to update shared data, it creates a copy, modifies the copy, and then atomically replaces the pointer. Old readers continue using the original until a "grace period" elapses. RCU

is heavily used in Debian's networking subsystem and VFS layer, giving near-zero overhead reads in scenarios with very frequent reading and rare writing.

2.2.6 Classical Synchronization Problems Debian Context

Silberschatz et al. (Chapter 7) present several classical problems that illustrate synchronization challenges. These abstract problems map directly to real subsystems in Debian 13:

Classical Problem	Debian 13 / Linux Kernel Analogue
Bounded-Buffer Problem	Models Debian's pipe subsystem and message queues (mqueue). Producers and consumers share a fixed-size ring buffer protected by a mutex, with counting semaphores tracking empty and full slots.
Readers-Writers Problem	Models concurrent filesystem access. Debian's VFS layer uses read-write locks so multiple processes can <code>stat()</code> or read a file simultaneously, while write operations require exclusive access.
Dining-Philosophers Problem	A model for deadlock and starvation. Debian's lock ordering conventions exist specifically to avoid the circular-wait scenario this problem illustrates.

2.2.7 The Four Necessary Conditions

Silberschatz et al. (Chapter 8) establish that a deadlock can arise only when all four of the following conditions hold simultaneously:

Mutual Exclusion: At least one resource must be held in a non-shareable mode.

Hold and Wait: A process holds at least one resource and is waiting to acquire additional resources held by other processes.

No Preemption: Resources cannot be forcibly taken away from a process; they must be released voluntarily.

Circular Wait: A set of processes $\{P_0, P_1, \dots, P_n\}$ exists such that P_0 waits for a resource held by P_1 , P_1 waits for one held by P_2 , and so on, with P_n waiting for a resource held by P_0 .

In Debian 13, the kernel does not implement a general-purpose deadlock detection algorithm at runtime for all locks. Instead, it relies primarily on deadlock prevention through strict lock ordering conventions combined with the `lockdep` tool.

2.2.8 Deadlock Prevention via Lock Ordering

Debian's Linux kernel prevents the circular-wait condition by imposing a total ordering on all lock acquisitions. Every lock type in the kernel is assigned to a lock class. The rule is simple but strictly enforced: if a thread needs multiple locks, it must always acquire them in the same predefined order. If lock A is always acquired before lock B throughout the kernel, circular wait involving only A and B is impossible. This is an example of deadlock prevention as defined by

Silberschatz et al. (Chapter 8): "We can prevent deadlock by ensuring that at least one of the four necessary conditions cannot hold."

2.2.9 The lockdep Tool

Debian 13 ships with the kernel's built-in lock dependency validator, lockdep, which can be enabled at compile time (`CONFIG_PROVE_LOCKING=y`). Silberschatz et al. (Chapter 8) note that Linux includes the lockdep tool for detecting potential deadlocks. At runtime, lockdep tracks the order in which every lock is acquired across every execution path. If it detects a potential cycle in the lock dependency graph indicating a possible circular wait it immediately prints a warning with the full call chain, even if an actual deadlock has not yet occurred. This allows kernel developers and system administrators to catch deadlock-prone code paths before they manifest in production.

2.2.10 Livelock

Silberschatz et al. (Chapter 8) distinguish deadlock from a related problem: livelock. In a livelock, processes are not blocked but continually change state in response to each other without making progress. In Debian's kernel, retry-based lock acquisition (e.g., using `spin_trylock()` in a loop) can theoretically cause livelock if two threads perpetually yield to each other. The kernel avoids this through randomized backoff and priority-aware scheduling.

2.2.11 Deadlock Detection and Recovery (User Space)

For user-space applications running on Debian 13, Java's thread dump mechanism (accessible via `kill -QUIT` or `jstack`) can identify deadlocks in JVM-based applications. The JVM will print a deadlock report listing the threads involved and the monitors they are waiting on. This aligns with what Silberschatz et al. describe as a deadlock detection algorithm allowing the system to identify and then potentially recover from deadlock by terminating one of the involved threads.

2.3 Memory Management

2.3.0. Introduction

Memory management is one of the most critical responsibilities of any operating system. Silberschatz et al. (Chapter 9) explain that the CPU can directly access only main memory and registers; thus, instructions and data must reside in memory during execution. The operating system must allocate and reclaim memory efficiently, isolate processes from one another, and present each process with the illusion of a large, contiguous address space even when physical memory is fragmented or insufficient.

Debian 13 "Trixie", running on the Linux 6.x LTS kernel, implements a sophisticated, multi-layered memory management system. This section examines its core mechanisms: virtual memory and address spaces, paging with page tables, the buddy system for physical page allocation, the SLUB allocator for kernel object allocation, and demand paging with page replacement.

2.3.1 Virtual Memory and Process Address Spaces

Debian 13 provides each process with a private virtual address space. On 64-bit (amd64) systems the primary architecture for Debian 13 the virtual address space is 128 TB for user space and 128 TB for kernel space (using a 48-bit address space). Silberschatz et al. (Chapter 9) describe virtual memory as a technique that "allows the execution of processes that are not completely in memory," enabling programs to be larger than physical memory and providing process isolation.

The key data structure for managing a process's virtual address space in Debian's kernel is the memory descriptor (struct `mm_struct`), which contains the page global directory (the top-level page table pointer), a red-black tree of Virtual Memory Areas (VMAs) each representing a contiguous region with uniform attributes, and metadata such as memory limits enforced by cgroups.

Memory regions within a process include: the text (code) segment (read-only, executable), the initialized and uninitialized data segments (BSS), the heap (grows upward), memory-mapped files and shared libraries, and the stack (grows downward). The kernel's `mmap()` system call creates new VMAs, for example when loading a shared library or performing file-backed memory mapping.

2.3.2 Paging: The Foundation of Memory Isolation

Debian 13's memory system is built entirely on paging. Silberschatz et al. (Chapter 9) define paging as a memory-management scheme that "permits the physical-address space of a process to be noncontiguous." The virtual address space is divided into fixed-size units called pages; physical memory is divided into frames of the same size. On Debian 13 (x86-64), the standard page size is 4 KB (4,096 bytes), though the kernel also supports Huge Pages of 2 MB and 1 GB for workloads that benefit from reduced TLB pressure.

2.3.3 Four-Level Page Table (x86-64)

For the x86-64 architecture used in most Debian 13 installations, the Linux kernel uses a four-level page table structure:

Level	Description
PGD (Page Global Dir.)	Level 4 top-level, one per process, pointed to by the CR3 register.
PUD (Page Upper Dir.)	Level 3 subdivides the PGD entry.
PMD (Page Middle Dir.)	Level 2 subdivides the PUD entry.
PTE (Page Table Entry)	Level 1 maps a 4 KB virtual page to a physical frame number and stores protection bits (Present, Read/Write, User/Supervisor, Accessed, Dirty).

When a process accesses a virtual address, the CPU's Memory Management Unit (MMU) walks the page table hierarchy to locate the corresponding physical frame. The Translation Lookaside Buffer (TLB) caches recent translations to avoid repeating this four-level walk on every memory access.

2.3.4 Page Faults

If a page is not present in physical memory, the MMU triggers a page fault exception. Silberschatz et al. (Chapter 10) explain that the OS handles this by bringing the required page into a free frame, updating the page table, and restarting the faulting instruction. Debian 13 distinguishes between two types of page faults:

- **Minor page faults:** The page exists in memory but is not yet mapped (e.g., anonymous pages, copy-on-write). No disk I/O is needed; only a page table update is required. These are extremely fast.
- **Major page faults:** The page must be fetched from swap space or a file on disk. These are expensive and involve I/O. Both counts are visible per process via `/proc/<pid>/stat`.

2.3.5 Physical Memory Allocation: The Buddy System

When the kernel needs to allocate one or more contiguous physical page frames, it uses the Buddy System allocator. Silberschatz et al. (Chapter 9) describe this as a system that "allocates memory from a fixed-size segment consisting of physically contiguous pages" using powers of two.

2.3.6 How the Buddy System Works

Debian 13's kernel maintains separate free lists for each memory order (order 0 through order 10, i.e., blocks of 1 to 1,024 contiguous pages). When a request arrives for 2^n pages:

The kernel looks for a free block of exactly 2^n pages. If found, it is returned immediately.

If no block of that size exists, a larger block ($2^{(n+1)}$ pages) is split into two equal halves the "buddies." One half satisfies the request; the other is placed on the free list.

When a block is freed, the kernel checks whether its buddy is also free. If so, the two buddies are merged (coalesced) back into a larger block, reducing fragmentation.

The status of the buddy allocator on a running Debian system is visible in `/proc/buddyinfo`, which shows the count of free blocks at each order for each memory zone.

2.3.7 Memory Zones

Debian 13's buddy allocator maintains separate free lists for each memory zone to accommodate hardware constraints:

Zone	Description
ZONE_DMA	First 16 MB for legacy ISA DMA devices requiring low physical addresses.
ZONE_DMA32	First 4 GB for 32-bit PCI devices on 64-bit systems.
ZONE_NORMAL	The main memory zone used for all regular kernel and user allocations.
ZONE_MOVABLE	Pages that can be migrated or reclaimed, used for memory compaction to reduce fragmentation over time.

2.3.8 Limitations and Mitigation

The buddy system has a well-known weakness: internal fragmentation. Requesting 33 pages requires a 64-page block (order 6), wasting 31 pages. Debian's kernel mitigates this via page mobility grouping (separating movable from unmovable pages) and Transparent Huge Pages (THP), which coalesce 4 KB pages into 2 MB huge pages for appropriate workloads, improving TLB hit rates substantially.

2.3.9. Kernel Object Allocation: The SLUB Allocator

The buddy system's granularity (minimum 4 KB) is far too coarse for the kernel's frequent allocation of small objects such as `task_struct`, `inode`, `dentry`, and `socket`. Silberschatz et al. (Chapter 9) describe the slab allocator as a solution designed to "eliminate fragmentation caused by allocations and deallocations" of fixed-size kernel objects. Debian 13 uses SLUB, the default and optimized slab allocator that replaced the original SLAB allocator in mainstream kernels.

2.3.10 Slab Allocator Principles

The slab allocator is built on the concept of an object cache. For each frequently used kernel data type, a dedicated cache is pre-populated with pre-initialized objects. When the kernel needs a

new instance of that type, it simply retrieves a ready-to-use object from the cache rather than allocating raw memory and re-running a constructor. When the object is freed, it is returned to the cache so the next allocation of the same type is instantaneous. The key terms are:

Cache: A named store for objects of a specific type or size (e.g., a cache for `task_struct`).

Slab: One or more physically contiguous pages from the buddy allocator, divided into uniform object slots. A slab can be Full, Partial, or Empty.

Object: A single instance of the cached type stored within a slab slot.

2.3.11 The SLUB Implementation in Debian 13

SLUB simplifies the original SLAB design by eliminating a separate per-slab descriptor structure. Instead, slab metadata is embedded directly into the struct page (or from kernel 5.17 onward, a dedicated struct slab). Each slab maintains a freelist a singly-linked list of free object pointers stored directly within the free slots themselves. SLUB also maintains per-CPU active slabs, so allocation and deallocation on the same CPU core require no locking a critical optimization on multi-core systems.

For generic small allocations of variable size, the kernel exposes `kmalloc()`, which routes the request to one of approximately 20 size-class caches ranging from 8 bytes to 4 MB. The state of all slab caches can be inspected on a running Debian 13 system with:

```
cat /proc/slabinfo    # or use the 'slabtop' command
```

2.3.12 Demand Paging and Page Replacement

Debian 13 uses demand paging: a process's pages are loaded into physical memory only when first accessed, not at process creation. Silberschatz et al. (Chapter 10) describe this as "lazy" loading the swapper never brings a page into memory unless that page is needed. The advantage is faster process startup and lower memory pressure, since unused code paths may never be loaded at all.

2.3.13 Copy-on-Write (COW)

When a process calls `fork()`, Debian's kernel does not immediately copy the parent's memory. Instead, both parent and child share the same physical pages, mapped read-only in both address spaces. Only when one process attempts to write to a shared page does the kernel allocate a new frame, copy the page content, and update the writing process's page table entry to point to the private copy. This Copy-on-Write (COW) mechanism is described by Silberschatz et al. (Chapter 10) and is critical to Debian's efficiency, particularly for servers that fork many short-lived child processes.

2.3.14 Page Replacement LRU Approximation

When free memory runs low, the kernel's page-out daemon (kswapd) must reclaim pages. Debian 13's kernel uses a variant of the clock (second-chance) algorithm refined as an LRU approximation. Each page frame has an Accessed bit set by the MMU whenever it is referenced. kswapd scans memory using separate active and inactive LRU lists:

Frequently accessed pages reside in the active list.

Pages not recently accessed are moved to the inactive list and become candidates for eviction.

Before evicting a page, its Accessed bit is checked. If set, the page is given a second chance and moved back to the active list; if clear, the page is reclaimed.

Modified (dirty) pages must be written back to swap space or the filesystem before the frame can be reused. Since Debian 13 ("Trixie"), the /tmp directory is stored in RAM by default using the tmpfs filesystem, reducing disk I/O for temporary files and improving performance.

2.3.15 Transparent Huge Pages and Memory Compaction

Debian 13 supports Transparent Huge Pages (THP), whereby the kernel automatically promotes groups of contiguous 4 KB pages into a single 2 MB huge page where possible. This reduces the number of TLB entries needed and improves performance for memory-intensive workloads. Memory compaction relocates movable pages to create larger contiguous free regions, enabling huge page promotion. Compaction is performed both proactively in the background and reactively when a huge page allocation fails.

2.3.16 Monitoring Memory Management on Debian 13

Debian 13 exposes detailed memory management information through the /proc virtual filesystem and dedicated command-line tools:

Tool / File	Information Provided
/proc/meminfo	Overview of total, free, available, cached, swap, and huge-page memory statistics.
/proc/buddyinfo	Free block counts per order per zone in the buddy allocator.
slabtop	Per-cache object counts, slab counts, and memory usage in the SLUB allocator.
vmstat	Virtual memory statistics including page-in, page-out, and swap rates.
free -h	Human-readable summary of total, used, free, and available memory.
/proc/<pid>/maps	Virtual memory areas of a specific process with addresses and permissions.

/proc/<pid>/status	Per-process memory usage including VmRSS (resident set size) and VmSwap.
--------------------	--

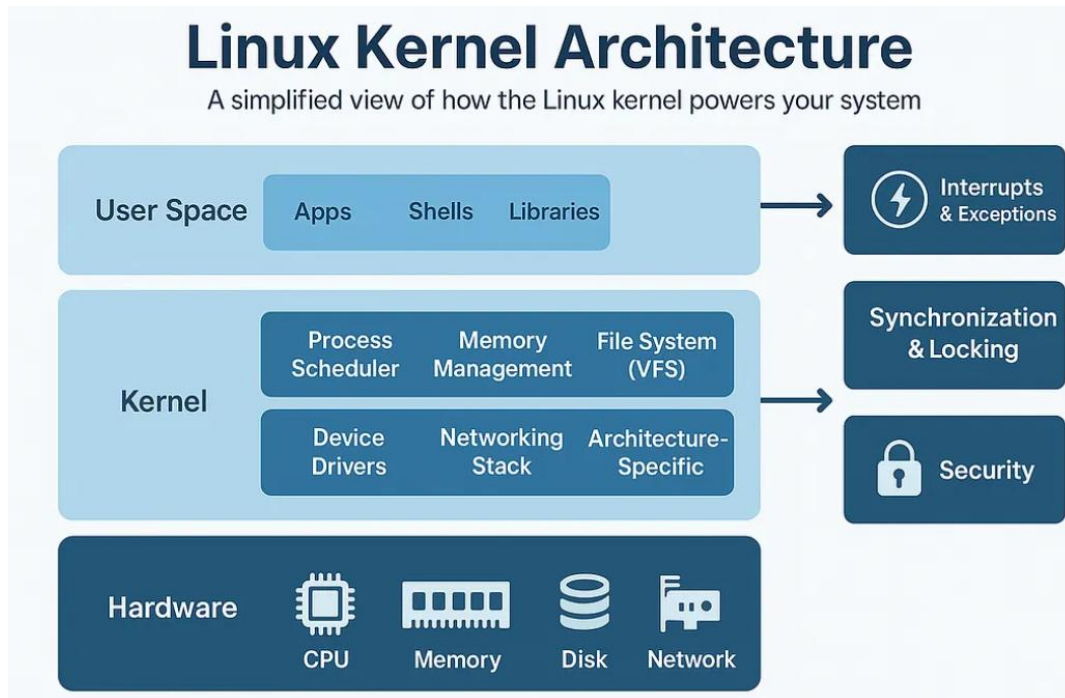
2.4 Kernel Architecture Evaluation

2.4.0 Introduction

Debian 13.4 is a Linux distribution built on the Linux kernel, and its operating-system behaviour is shaped by the way this kernel is structured and how it enforces protection over system resources. A meaningful analysis of Debian therefore requires attention to both the internal architecture of the kernel and the security model through which the operating system manages access, privilege, and control.

Kernel architecture is central because it defines how the operating system organizes its core functions, how system services are executed, and how the kernel interacts with applications and hardware. In Debian, these functions are handled through the Linux kernel, which provides process control, memory management, filesystem support, device interaction, and privileged execution. At the same time, the security model of Debian depends on this architecture because the kernel is the component that enforces privilege boundaries, validates access rights, and supports security extensions.

This report focuses on two areas. The first is kernel architecture evaluation, which examines the structural design of the Linux kernel used by Debian, including its monolithic nature, modular capabilities, subsystem organization, and overall strengths and limitations. The second is the operating-system security model, which examines how Debian controls access and protects resources through privilege separation, user and group permissions, authentication, and kernel-supported security frameworks. Together, these two areas provide a clear view of Debian 13.4 as



a practical Linux-based operating system.

Figure 1. Linux kernel architecture showing the relationship between user space, kernel subsystems, and hardware components.

2.4.1 The Linux Kernel as the Core of Debian

The Linux kernel is the central component of Debian 13.4. It forms the core execution environment through which protected operating-system services are provided. Applications, utilities, and user-level programs operate outside the kernel, while privileged functionality remains inside the kernel itself. This separation is important because it allows the system to maintain control over hardware access, resource allocation, and critical system behavior.

In Debian, the kernel does not function as an isolated technical layer without surrounding support. Instead, it exists within a larger system that includes user-space applications, shared libraries, system tools, and administration utilities. Even so, the kernel remains the authoritative controller of the operating system. Tasks such as scheduling, memory control, device communication, and protected access to files or hardware are handled through the kernel's privileged mechanisms.

This makes the kernel the architectural foundation of Debian. It is the component that connects software activity to hardware resources while maintaining a controlled operating environment. For that reason, any evaluation of Debian's operating-system structure must begin with the Linux kernel itself.

2.4.2 Monolithic Structure with Modular Design

One of the most important characteristics of the Linux kernel is that it combines a monolithic execution model with modular extensibility. In structural terms, the kernel operates as one large, privileged binary in a single address space. Core services are not separated into isolated server processes. Instead, kernel components operate within one protected execution domain, which improves efficiency and allows subsystems to interact directly through internal function calls.

This monolithic structure offers a clear performance advantage. Since kernel services reside in one privileged space, communication among subsystems is faster and more direct than it would be in a design that depends heavily on message passing or repeated context switching between separate system components. This helps Linux maintain strong execution efficiency in process control, memory handling, filesystem operations, and driver interaction.

At the same time, Linux is not architecturally rigid. The kernel supports dynamically loadable modules, which means that many features can be compiled and managed separately rather than permanently built into the kernel image. This gives Debian flexibility in supporting drivers, kernel features, and hardware-related extensions. It also improves maintainability, because the operating system can support a broad range of hardware and functions without forcing all features into the fixed base kernel.

This combination makes Debian's kernel architecture especially effective. It preserves the speed and directness of a monolithic kernel while also supporting the adaptability expected in modern operating systems.

Table 1. Structural Characteristics of the Debian Kernel

Architectural Aspect	Description	Significance in Debian
Monolithic execution model	Core kernel services operate in one privileged address space	Improves efficiency and direct subsystem interaction
Modular design	Features can be added or removed as loadable modules	Increases flexibility and hardware support
Privileged kernel control	Critical operations are performed inside the kernel	Strengthens centralized resource management
Structured subsystem layout	The kernel is divided into specialized implementation areas	Supports clarity, maintainability, and functional separation

2.4.3 Kernel Mode, User Mode, and the System-Call Interface

A central feature of Debian's kernel architecture is the separation between kernel mode and user mode. Kernel mode is the privileged execution level in which the operating system performs its core functions and has unrestricted access to protected resources. User mode, by contrast, is the restricted environment in which ordinary applications and user programs operate.

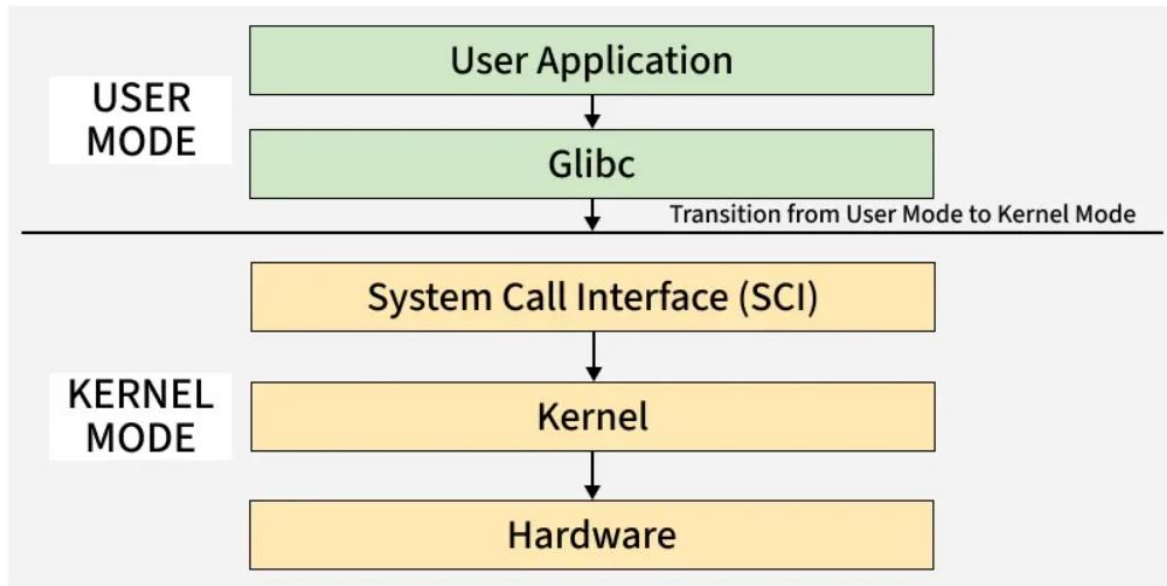
This separation is essential to the structure of Debian because it prevents user applications from directly controlling memory, hardware, or sensitive system resources. Instead, applications must use formal operating-system interfaces to request services. These requests are transferred to the kernel through system calls, which act as controlled entry points into privileged execution.

The system-call interface is therefore a bridge between user space and kernel space. It allows user programs to access operating-system services without bypassing the kernel's control.

This arrangement improves both order and protection. It preserves the role of the kernel as the system's central authority while allowing ordinary software to function within clear limits.

From an architectural perspective, this model is one of Debian's strongest design features. It provides a clean division of responsibility, keeps protected operations centralized, and makes the kernel the controlled gateway to system functionality.

Figure 2. User mode to kernel mode transition through the system-call interface.



2.4.4 Major Internal Kernel Areas Relevant to Debian

The Linux kernel used by Debian is not a single undivided block of logic. It is internally organized into specialized areas responsible for different aspects of operating-system behavior. These include core execution logic, device drivers, memory management, filesystems, networking, security-related components, and virtualization support.

This internal organization is important because it shows that Debian's kernel is structured according to functional responsibility. Memory management is handled in dedicated areas rather than being mixed randomly with driver logic. Filesystem support is represented separately from process control. Security-related functionality is also assigned its own implementation space. This reflects a kernel architecture that is broad in capability but organized in a disciplined manner.

Such internal specialization improves maintainability and helps support real operating-system complexity. Debian benefits from this structure because it allows the distribution to build

on a kernel that is both large and manageable. Each internal area contributes to the larger operating-system environment without eliminating the overall unity of the kernel.

Table 2. Major Kernel Areas Relevant to Debian

Kernel Area	Main Responsibility	Relevance to Debian
Core kernel logic	Scheduling, process handling, and system control	Supports overall operating-system execution
Memory management	Allocation, paging, and virtual memory handling	Essential for stability and performance
Filesystem support	Filesystem interfaces and storage interaction	Supports file access and data management
Device drivers	Communication with hardware devices	Enables hardware compatibility
Networking	Network-related kernel functions	Supports communication and connectivity
Security support	Security hooks and kernel-level protections	Directly supports the OS security model
Virtualization support	Infrastructure for virtualized environments	Relevant to VM-based deployment and analysis

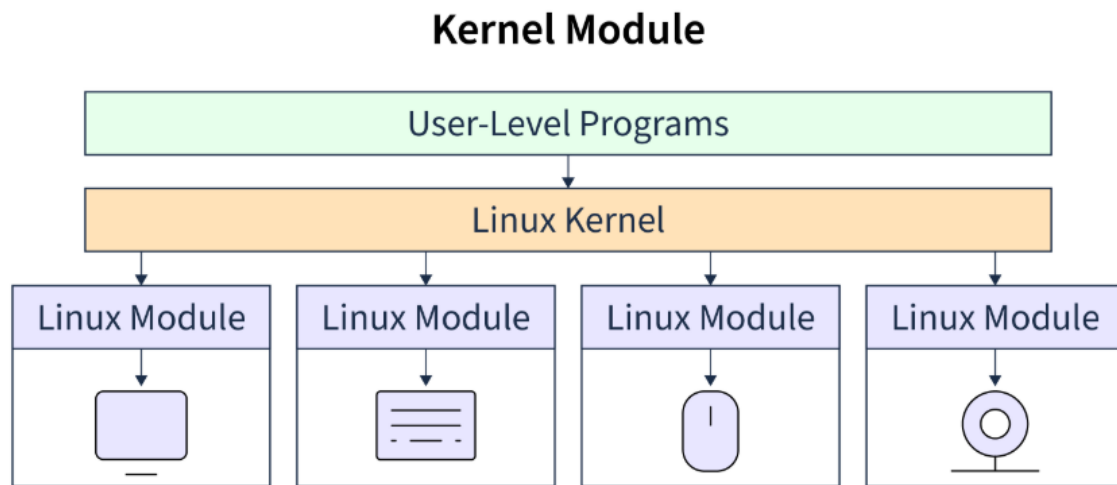
2.4.5 Kernel Modules and Debian Maintainability

Kernel modules are one of the most important practical features of the Linux kernel used in Debian. A module is a kernel component that can be loaded or unloaded separately rather than being permanently embedded in the base kernel image. This gives Debian a more flexible operating-system structure, especially in relation to hardware support and optional features.

The modular model supports real-world maintainability. Rather than forcing every feature into one fixed kernel build, Debian can use modules to support a wide variety of drivers and kernel extensions. This is especially useful in systems that need to support different hardware environments without rebuilding the entire operating system each time a change is required.

From an evaluative perspective, kernel modules are one of Debian's strongest architectural assets. They allow the operating system to remain adaptable while still preserving the efficiency of a monolithic execution model. However, modules also introduce complexity. They require compatibility management, correct packaging, and careful integration with the active kernel version. As a result, modularity improves flexibility, but it also increases administrative responsibility.

Figure 3. Loadable kernel modules in Linux showing how additional components can be



dynamically integrated with the kernel.

2.4.6 Evaluation of Debian Kernel Architecture

Debian's kernel architecture can be evaluated as efficient, structured, and highly adaptable. Its monolithic execution model supports direct subsystem cooperation and reduces communication overhead. Its modular capabilities provide flexibility and broad hardware support. Its clear distinction between kernel mode and user mode preserves controlled access to protected resources. Its specialized internal layout gives the kernel a functional structure that supports maintainability and growth.

The main strengths of this architecture are performance, centralized resource control, modular extensibility, and well-defined subsystem responsibilities. These features make Debian suitable for a wide range of computing environments and explain why Linux distributions remain widely used in both desktop and server contexts.

At the same time, this architecture also has limitations. Because the kernel operates in a privileged address space, faults at kernel level can have system-wide effects. The same tight

integration that improves efficiency also increases the importance of correctness, reliability, and security in kernel code. Module management, while useful, also adds complexity.

Overall, Debian's kernel architecture represents a balanced and mature design. It combines strong execution efficiency with operational flexibility, but it depends on disciplined maintenance and careful system management.

Table 3. Evaluation Summary of Debian Kernel Architecture

Criterion	Strength	Limitation
Performance	Direct subsystem interaction improves efficiency	Kernel-level faults can affect the whole system
Modularity	Supports flexibility and broad hardware compatibility	Requires compatibility and version management
Maintainability	Structured subsystem organization supports clarity	Large kernel complexity increases administrative burden
Resource control	Privileged kernel enforces centralized control	Strong kernel dependence increases risk concentration

2.5 Security Model

2.5.0 Introduction

The operating-system security model of Debian is built around controlled access to resources, separation of privileges, and enforcement through the kernel. Security in this context is not only a matter of passwords or user accounts. It is a broader system of rules and protections through which the operating system limits access, preserves integrity, and prevents unauthorized control.

The kernel is central to this design because it is the privileged component that enforces protection. Since ordinary user programs do not run with the same privilege level as the kernel, the operating system can maintain boundaries between restricted and unrestricted actions. This means that Debian's security model depends heavily on the same architectural separation discussed earlier.

Security in Debian is therefore layered into the operating system itself. It is not external to the system's architecture. Instead, it is built through privilege separation, controlled interfaces, access rules, authentication support, and additional kernel-level enforcement mechanisms.

2.5.1 User and Group Based Access Control

A fundamental part of Debian's security model is the use of user identifiers, group identifiers, and permission masks to control access to system objects. Files and other protected objects are associated with ownership information, and processes are evaluated according to their identity and privilege level. This creates a consistent method for deciding whether a process should be allowed to read, modify, or execute a given resource.

This model provides a practical and recognizable structure for operating-system protection. It distinguishes between the object owner, members of an associated group, and all other users. It then assigns read, write, and execute rights to those categories. This approach is widely used in Linux-based systems because it is simple, direct, and effective in many common operating scenarios.

An especially important aspect of this model is the privileged status of the root account. Root-level privileges allow processes to bypass many ordinary access restrictions. This makes administration possible, but it also creates risk if privileged access is granted too broadly or used unnecessarily. For that reason, Debian's security model is strongest when permissions are carefully limited, and high privilege is used only where necessary.

Table 4. Core Access-Control Elements in Debian

Security Element	Function	Security Role
User identity	Distinguishes one user from another	Enables owner-based control
Group identity	Represents shared access categories	Supports collaborative access management
Permission mask	Defines read, write, and execute rights	Controls object-level access

Root privilege	Grants elevated administrative authority	Supports administration but increases risk if overused
----------------	--	--

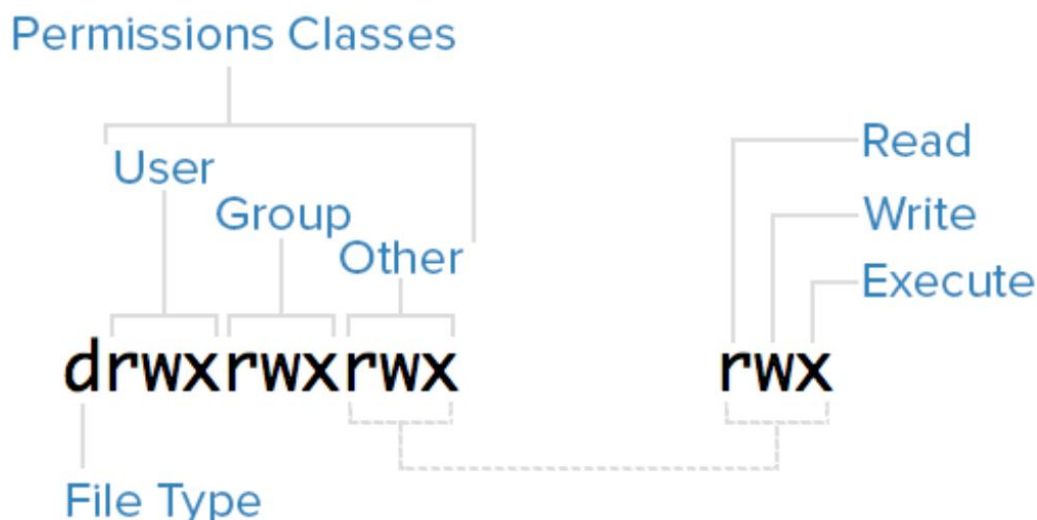


Figure 4. Linux file-permission model showing user, group, and others along with read, write, and execute access rights and file type representation.

2.5.2 Authentication and Pluggable Mechanisms

Debian's security model also includes an authentication layer that can support multiple methods and account-related controls. A key feature of Linux-based systems in this area is the use of pluggable authentication mechanisms. This approach allows authentication behaviour to be adjusted through system configuration rather than requiring the entire operating system to be redesigned whenever a new method is introduced.

This design improves flexibility and administration. It allows the operating system to support different authentication requirements while keeping a structured framework for login control, account restriction, session handling, and password-related operations. It therefore strengthens Debian's security model by separating authentication policy from the deeper architectural core of the system.

However, the same flexibility also increases the importance of correct configuration. Authentication frameworks are only effective when the administrative policy that governs them is carefully managed. In that sense, Debian's design is strong because it is adaptable, but it still depends on correct deployment.

2.5.3 Least Privilege, Compartmentalization, and Defence in Depth

Debian's security model reflects three important security principles: least privilege, compartmentalization, and defence in depth. Least privilege means that users, processes, and programs should be granted only the access required to perform their intended functions. Compartmentalization means that different parts of the system should be isolated so that one component does not automatically gain unrestricted access to others. Defence in depth means that security should rely on multiple protective layers rather than one single mechanism.

These principles help explain why Debian uses several different controls rather than only one. File permissions, account identity, privileged mode separation, authentication controls, and kernel-level security frameworks all contribute to the larger security structure. Each layer adds protection, and together they provide a more resilient model than a single control could provide by itself.

This is one of the strongest features of Debian's security design. Its protection model is not dependent on only one rule or one feature. It is based on a layered operating-system logic in which multiple barriers cooperate to restrict access and reduce risk.

2.5.4 Linux Security Modules and Kernel-Level Security Enforcement

An important advanced feature of Debian's Linux-based security model is support for Linux Security Modules. This framework provides kernel-level hooks through which additional security models can be integrated into the operating system. It reflects the idea that security enforcement can be extended beyond standard discretionary permissions into more specialized or fine-grained control structures.

This is significant because it shows that Debian's security design is not limited to traditional ownership and permission masks. The operating system also has a kernel-supported framework for integrating stronger security logic where needed. This adds another layer to the security model and demonstrates that Linux-based security can be extended at the architectural level rather than only through user-level utilities.

The presence of security-related implementation areas in the kernel also supports the view that security is structurally embedded in the operating system. Debian's security model therefore includes both traditional access control and deeper kernel-level extensibility.

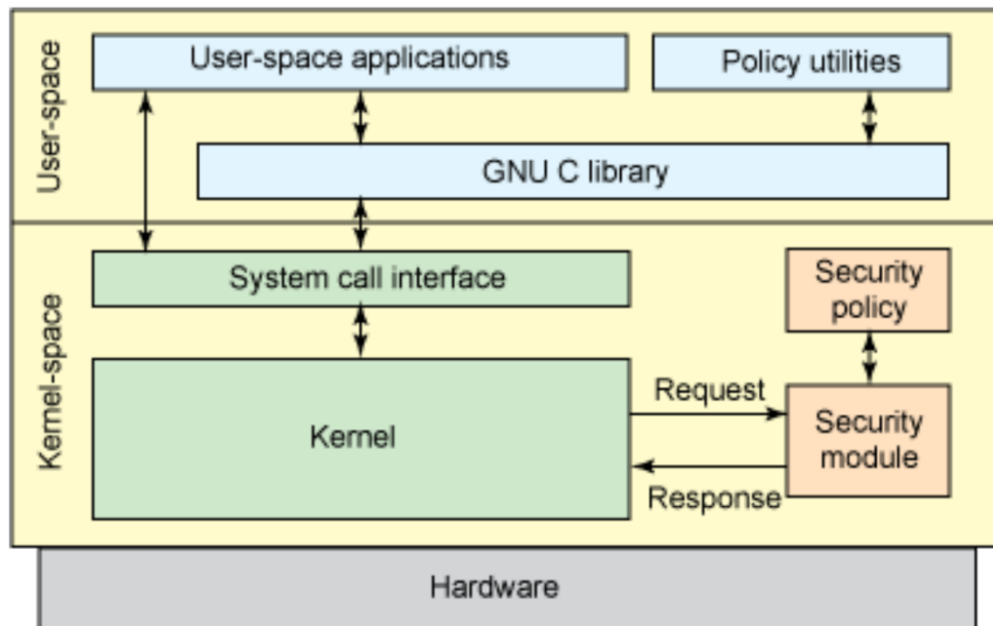


Figure 5. Linux Security Modules (LSM) framework showing interaction between user space, kernel, and security modules for enforcing security policies.

2.5.5 Protection Rings and Privileged Separation

Another important feature of Debian's security model is the separation between privileged and non-privileged execution. The operating system depends on the fact that the kernel runs with higher privilege than ordinary processes. This separation allows the system to protect memory, hardware control, and sensitive operations from direct user-level access.

This is not simply an implementation detail. It is one of the operating system's fundamental protection mechanisms. The division between privileged and restricted execution helps prevent unauthorized control and ensures that protected resources remain under kernel supervision. It also means that security in Debian begins at the architectural level and is then reinforced by higher-level controls.

Because of this, Debian's security model is strongest when privilege boundaries are respected, and administrative power is used carefully. The operating system can enforce

restrictions effectively, but the value of that enforcement depends on how privileges are granted and managed.

2.5.6 Evaluation and Concluding Thoughts of Debian's OS Security Model

Debian's operating-system security model can be evaluated as layered, structured, and strongly connected to kernel-enforced control. Its major strengths include the clear separation between kernel mode and user mode, the consistent use of user and group based access control, the support for configurable authentication, and the ability to extend security enforcement through Linux Security Modules.

These features create a security model that is practical and scalable. It supports ordinary multi-user operation while also allowing more advanced administrative control and deeper security integration. The use of multiple protective layers also makes the system more resilient than a design that depends on one mechanism alone.

Its limitations arise mainly from administration and configuration rather than from the absence of security features. High privilege can bypass ordinary controls, and flexible authentication or module-based security must be configured correctly to be effective. This means Debian's security design is strong in structure, but its real effectiveness depends on proper use.

Overall, Debian provides a well-developed security model that combines traditional operating-system protection with kernel-level extensibility and layered control.

Table 5. Evaluation Summary of Debian's OS Security Model

Criterion	Strength	Limitation
Privilege separation	Protects core resources through kernel/user distinction	Kernel-level weaknesses have broad impact
Access control	Provides clear user, group, and permission-based regulation	Root privileges can weaken ordinary restrictions
Authentication	Supports flexible and configurable control mechanisms	Effectiveness depends on correct configuration

Kernel security extensions	Enables stronger security integration at kernel level	Increased flexibility adds policy complexity
Layered protection	Multiple mechanisms strengthen overall security	Weak management in one layer can reduce total protection

Analysis of Debian 13.4 shows that kernel architecture and operating-system security are deeply interconnected. The Linux kernel provides the structural foundation of Debian, controlling access to hardware, managing privileged execution, and organizing the system through a combination of monolithic efficiency and modular flexibility. This architecture gives Debian strong performance, clear subsystem organization, and broad extensibility.

The security model builds directly on that foundation. Debian protects resources through privilege separation, user and group-based permissions, configurable authentication mechanisms, and kernel-level security extensibility. Rather than depending on one isolated feature, the system uses a layered protection approach that supports both ordinary operating needs and stronger forms of system control.

Together, these characteristics make Debian 13.4 a strong example of a Linux-based operating system in which structural design and security enforcement are closely integrated. Its main advantages are efficiency, flexibility, and layered protection. Its main challenges are complexity and the need for careful administration. Even so, the overall design remains highly effective and shows why Debian is widely recognized as a stable and technically mature Linux distribution.

3. Legal and Ethical Judgment in OS

3.1 Intellectual Property

Debian has gone through multiple releases and updates over time, with Debian 13.4 “Trixie” being the latest version (Nestor, 2026). These updates play a huge role in improving security, fixing bugs, and ensuring smoother overall performance of the system.

Debian 13.4 is a Linux-based operating system, and unlike many software systems that restrict users from certain actions, it is licensed under the GNU General Public License (GPL), making it free and open-source. This means that it allows all users to freely use it, distribute it or

even modify it (*Debian -- Reasons to Use Debian*, n.d.). To maintain the same level of freedom, users who redistribute the operating system must provide the same rights and freedom to the user receiving that version of the software. Although the main part of Debian is open-source, there are some hardware devices that require proprietary devices which are not open-source therefore they cannot be modified and their source code cannot be accessed. In order to avoid any confusion, Debian 13.4 segregates the open-source main section from the proprietary firmware to ensure transparency in the legal aspect of the system.

3.2 User Privacy

Given that Debian 13.4 is an open-source operating system, it is strongly committed to openness and freedom. Therefore, interactions such as the bug tracking system and emails received are publicly archived which means that they are available for public access. While this encourages collaboration and accountability within the community, it also means that any information shared through these interactions may be visible to others.

Nonetheless, Debian's privacy policy states that no registration or personal identification is required to download or use the operating system, meaning users are not obligated to provide any personal information (*Debian - Privacy Policy*, n.d.). This highlights Debian's user-focused approach, where privacy is preserved by default through minimal data collection.

However, due to these lenient and open policies, the default system configuration may not be sufficient for users who require stronger security and privacy protection. Therefore, it is important for users to take additional precautions, such as implementing strict firewall rules, ensuring updates come from reliable sources, and actively monitoring system activity. By taking these measures, users can significantly reduce the risk of unauthorized or untrusted access and potential security threats, making the system both secure and in line with their privacy needs.

3.3 System Security

Debian 13.4 manages to maintain a solid and strong system security with regular updates to patch any security issues, for instance, the last release had 67 security updates (Nestor, 2026). Debian always provides a "stable" version, which is the most reliable release and includes all necessary updates and security patches, with the current version being "Trixie."

The security continues to provide support for each stable release for approximately three years, along with an additional support period even after a newer stable version is released

(*DebianReleases* — *Debian Wiki*, n.d.). In addition to the stable release, Debian also offers “testing” and “unstable” development versions, which are continuously updated and play a key role in developing future stable releases.

This constant maintenance is very important and crucial for ensuring system reliability, stability, and long-term security, making Debian a trustworthy operating system for a wide range of users.

3.4 Accountability and Governance

One of the more distinctive aspects of Debian 13.4 from an ethical standpoint is the question of accountability, to be more specific, who is responsible when something goes wrong. Unlike commercial operating systems backed by a single corporation with defined legal liability, Debian is developed and maintained by the Debian Project, a globally distributed community of volunteer developers. There is no central corporate entity that bears legal responsibility for the system’s behaviour, which raises important considerations about transparency, governance, and trust.

Debian addresses this through two foundational documents: the Debian Social Contract and the Debian Constitution. The Social Contract is a public commitment made by the Debian Project to its users and to the free software community, outlining the project’s core values and obligations (*Debian Social Contract*, 2022). The Debian Constitution, on the other hand, formally defines the governance structure of the project, including how decisions are made, how disputes are resolved, and what authority different bodies hold within the organization (*Debian Constitution*, 2022). These documents serve as a form of institutional accountability, making the decision-making process transparent and accessible to anyone.

On the technical side, the Debian Security Team plays a key role in accountability by actively monitoring vulnerabilities and issuing Debian Security Advisories (DSAs) whenever a confirmed vulnerability is found in the distribution. Each DSA is publicly published with a detailed description of the vulnerability, the affected packages, and the corrective action taken (*Debian -- Security Information*, 2026). This process ensures that users are not kept in the dark about risks in their system, and it creates a traceable and auditable record of how security issues are handled.

Debian’s public bug tracking system (BTS) further reinforces accountability. Every reported bug, whether it relates to functionality, security, or behaviour, is logged publicly and

assigned to a maintainer who is responsible for addressing it. This creates a culture of transparent responsibility where inaction or poor handling of issues is visible to the entire community. While this open model differs from the contractual accountability of a commercial vendor, it achieves a comparable level of traceability and community oversight, which in many ways strengthens trust in the system.

3.5 Responsible Use of Computing Resources

Responsible use of computing resources is an ethical principle that concerns how an operating system is designed to be used, what it enables by default, and how it guides users toward safe and appropriate behaviour. Debian 13.4 reflects a clear and deliberate philosophy in this regard, one that prioritizes user freedom while building in mechanisms that encourage responsible conduct at both the system and user level.

By default, Debian 13.4 ships with no telemetry, no background data reporting, and no advertising frameworks embedded in the system. This design choice means that the operating system does not consume network or computational resources for purposes beyond what the user explicitly initiates. This stands in contrast to several commercial operating systems that collect diagnostic and usage data in the background, often without the user's full awareness. Debian's approach ensures that system resources are used solely for the tasks the user intends, reflecting an ethical commitment to non-exploitative design.

Debian's package repository structure also reflects responsible use principles. The main repository contains only free and open-source software that complies with the Debian Free Software Guidelines. Proprietary software is separated into the non-free and contrib repositories, and access to these must be explicitly enabled by the user. This opt-in model is ethically significant: rather than silently bundling software with unclear licensing or behavioral implications, Debian requires users to make a conscious, informed decision before installing such software (*Debian -- What Does Free Mean?*, 2024). This design places responsibility on the user while ensuring they are always aware of what they are choosing.

At the system level, Debian enforces a strict privilege model to ensure that computing resources are not misused or accessed without authorization. Regular users operate with limited permissions by default, and elevated privileges are granted only through the sudo mechanism, which requires explicit configuration and authentication. This means that potentially damaging

or system-wide operations cannot be performed accidentally or without deliberate intent, reducing the risk of resource misuse, whether through user error or malicious software (*Sudo - Debian Wiki*, 2026).

Finally, it is worth noting the broader deployment context of Debian 13.4. The operating system is widely used in servers, embedded systems, research environments, and critical infrastructure, which amplifies the ethical weight of its design choices. The fact that Debian prioritizes stability, long-term support, and minimal default resource consumption makes it well-suited for environments where misuse or instability could have serious consequences. This responsible-by-design approach reflects an awareness that an operating system's ethical responsibilities extend beyond the individual user to the systems and services that depend on it.

4. Implementation

4.1 Virtual Machine Setup

Selected Distribution: Debian GNU/Linux

Virtualization Platform: VirtualBox

Overview

This section documents the configuration and initialization of the virtual machine environment used to host the Debian GNU/Linux operating system. Oracle VirtualBox was selected as the hypervisor due to its open-source nature, cross-platform availability, and robust support for Linux guests. VirtualBox operates as a Type 2 hypervisor, meaning it runs on top of a host operating system and emulates hardware resources for the guest environment. This arrangement allows safe observation and experimentation without risk to the underlying host system.

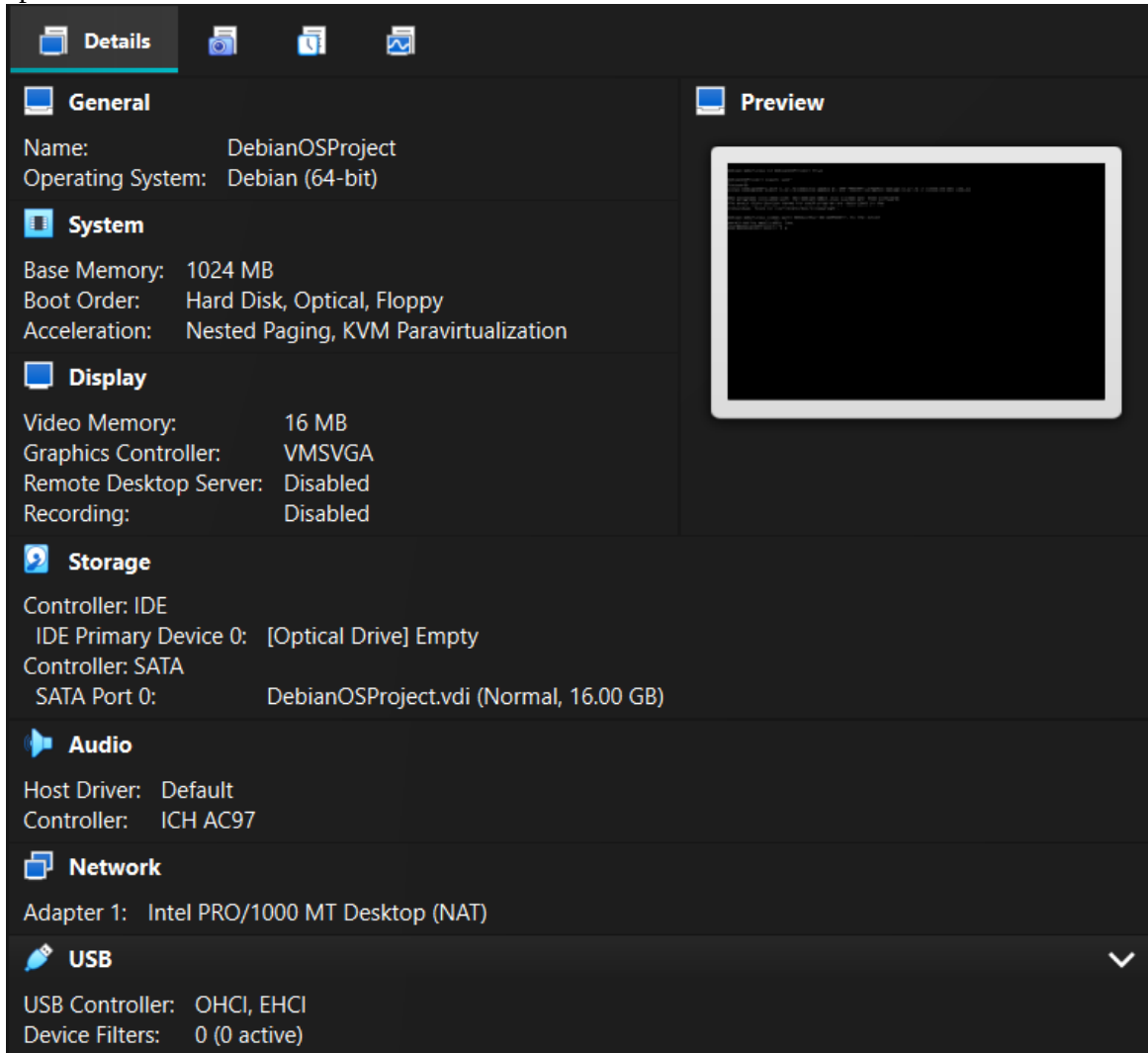
Hardware Configuration

Prior to installing Debian, a new virtual machine was created within VirtualBox with the following specifications:

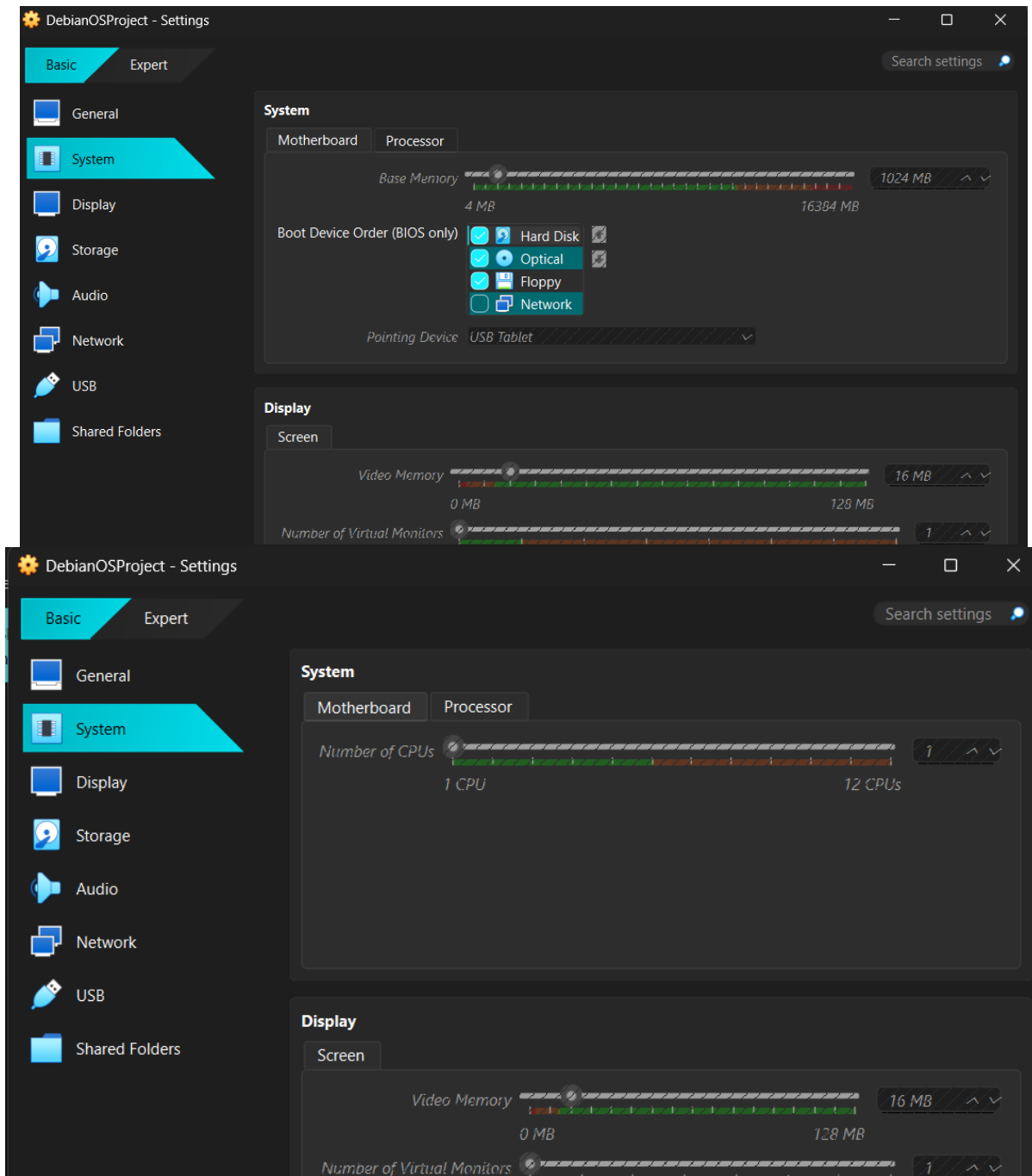
- **Type:** Linux
- **Version:** Debian (64-bit)
- **Base Memory (RAM):** 1024 MB
- **Virtual CPU Cores:** 1 core
- **Virtual Hard Disk:** 16 GB (VDI format, Dynamically Allocated)

- **Network Adapter:** NAT (Network Address Translation)
- **Display Memory:** 16 MB

These settings were chosen to provide just enough sufficient resources to run monitoring tools and perform stress tests.



VirtualBox Manager window showing the newly created VM with configured hardware settings.



VM System settings tab displaying CPU and memory allocations.

Notes on Configuration Choices

The use of dynamically allocated storage ensures that the virtual disk only consumes host disk space proportional to actual usage, making the setup efficient. NAT networking was preferred at this stage to allow internet access for package installation without exposing the guest to the host network directly.

4.2 Operating System Installation

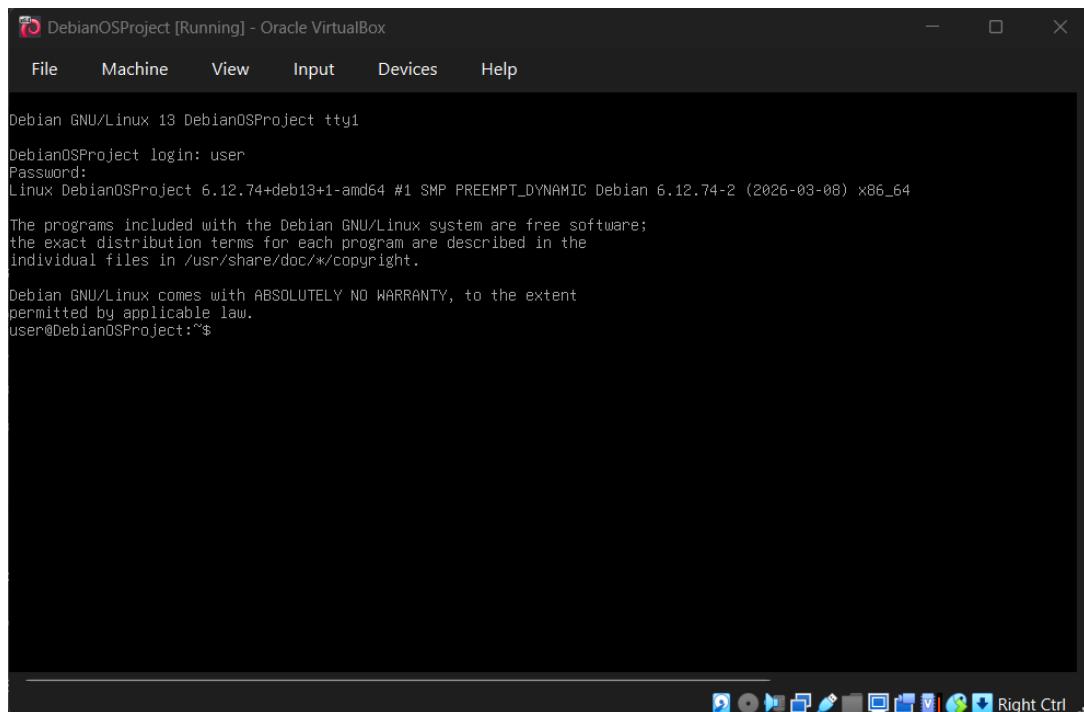
Overview

Debian GNU/Linux is one of the oldest and most widely respected Linux distributions, serving as the upstream base for many other distributions including Ubuntu. Debian emphasizes stability, free software principles, and a robust package management system (APT/dpkg). The version used in this case study is **Debian 13.4 Trixie**, installed from an official ISO image obtained from the Debian project website (<https://www.debian.org>).

Installation Process

The installation was carried out using the Debian **netinst** (network installer) image, which provides a minimal base system and downloads additional packages from the internet during setup. The following steps were performed:

1. The ISO image was mounted as a virtual optical disk in VirtualBox.
2. The VM was booted, and the Debian installer was launched.
3. A root password and a standard user account were created.
4. After reboot, the system was confirmed to be running and accessible.

A screenshot of a terminal window titled "DebianOSProject [Running] - Oracle VirtualBox". The terminal shows the login process for a user named "user". The prompt is "user@DebianOSProject:~\$". The terminal output includes the Debian version "6.12.74+deb13+1-amd64" and the kernel version "6.12.74-2 (2026-03-08) x86_64". The terminal also displays the Debian GNU/Linux license information, stating that the programs are free software and that the system comes with absolutely no warranty.

```
Debian GNU/Linux 13 DebianOSProject tty1
DebianOSProject login: user
Password:
Linux DebianOSProject 6.12.74+deb13+1-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.74-2 (2026-03-08) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

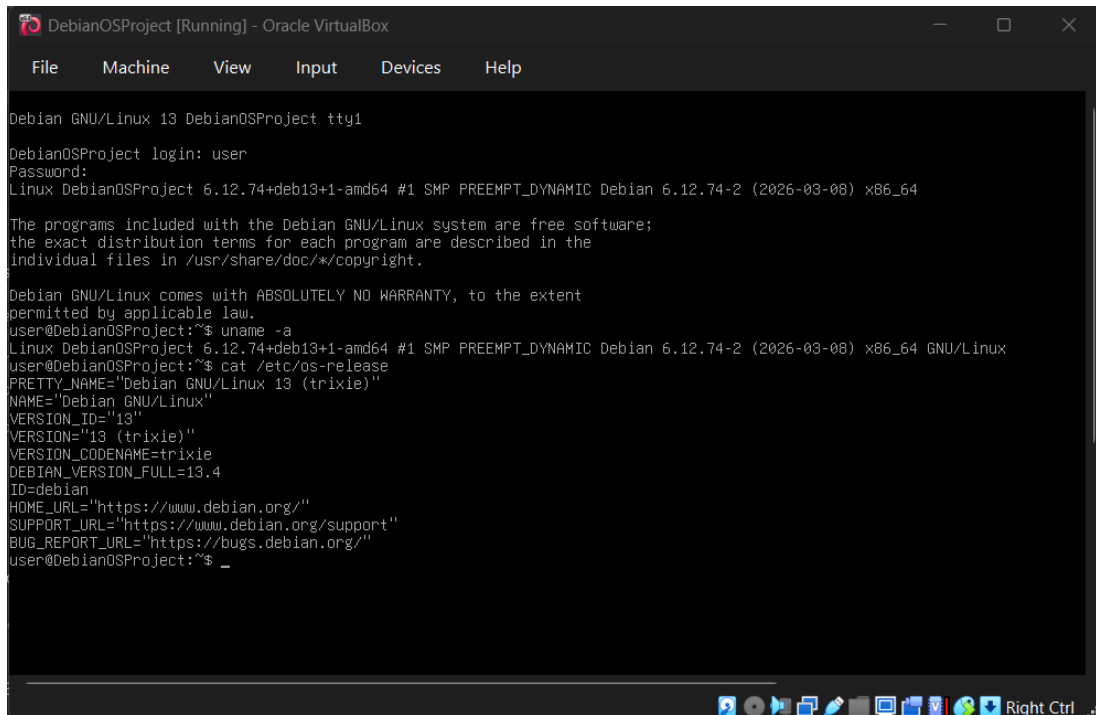
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
user@DebianOSProject:~$
```

Debian terminal prompt after successful first boot, confirming installation completion.

Post-Installation Verification

After installation, the following command was used to confirm the Debian version and kernel information:

```
uname -a
cat /etc/os-release
```



```
Debian GNU/Linux 13 DebianOSProject tty1
DebianOSProject login: user
Password:
Linux DebianOSProject 6.12.74+deb13+1-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.74-2 (2026-03-08) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
user@DebianOSProject:~$ uname -a
Linux DebianOSProject 6.12.74+deb13+1-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.74-2 (2026-03-08) x86_64 GNU/Linux
user@DebianOSProject:~$ cat /etc/os-release
PRETTY_NAME="Debian GNU/Linux 13 (trixie)"
NAME="Debian GNU/Linux"
VERSION_ID="13"
VERSION="13 (trixie)"
VERSION_CODENAME=trixie
DEBIAN_VERSION_FULL=13.4
ID=debian
HOME_URL="https://www.debian.org/"
SUPPORT_URL="https://www.debian.org/support"
BUG_REPORT_URL="https://bugs.debian.org/"
user@DebianOSProject:~$ _
```

Terminal output of `uname -a` and `cat /etc/os-release` confirming the installed Debian version and Linux kernel version.

4.3 Installation of System Monitoring and Tracing Tools

Overview

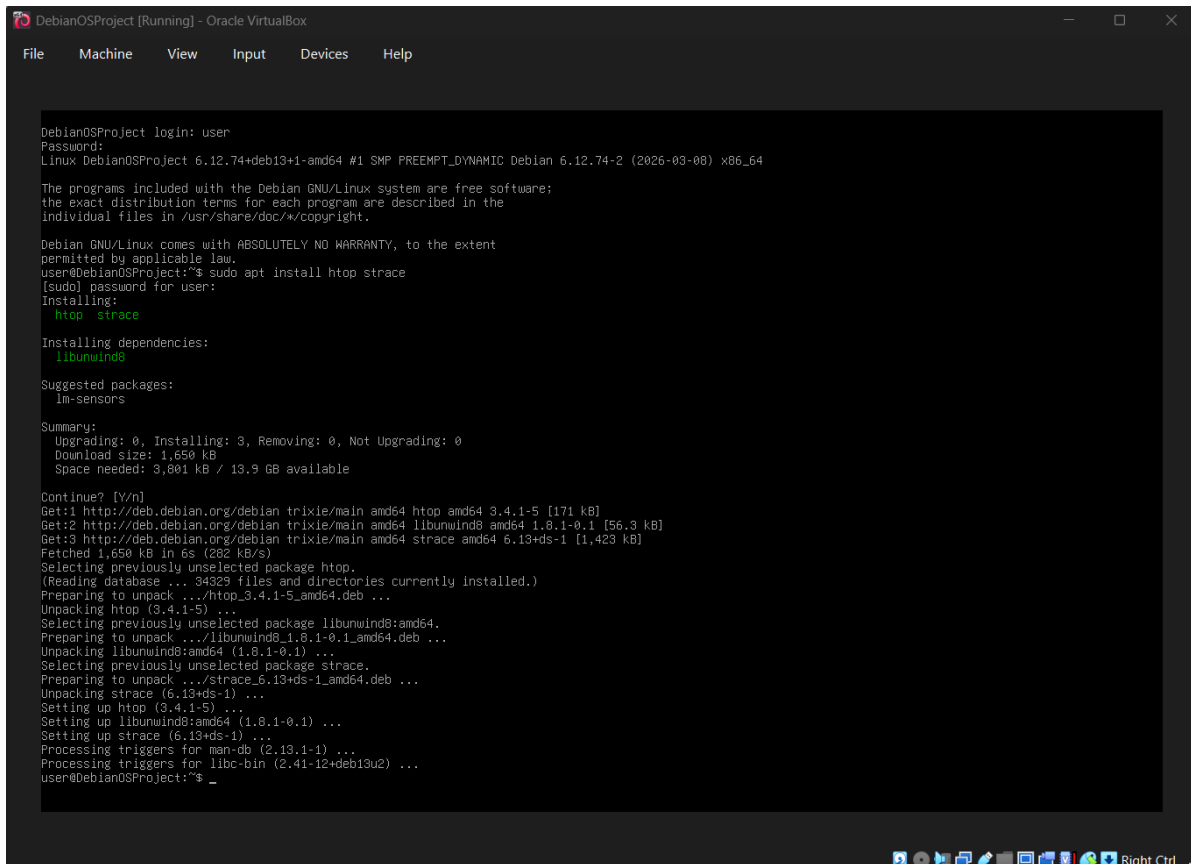
To facilitate the analysis of process behavior, memory usage, and system calls, three standard Linux tools were installed and configured: `top`, `htop`, and `strace`. Each tool provides a different layer of visibility into the operating system's runtime behavior.

4.3.1 Installing the Tools

All tools were installed using Debian's Advanced Package Tool (APT). The following command was executed with superuser privileges:

`sudo apt install htop strace`

Note: `top` is part of the `procps` package, which is included in the Debian base installation by default and therefore did not require a separate installation step.

A screenshot of a terminal window titled "DebianOSProject [Running] - Oracle VirtualBox". The terminal shows a user logging in and running the command "sudo apt install htop strace". The output displays the installation process, including downloading packages, unpacking, and setting up. The packages installed are htop (3.4.1-5) and strace (6.13+ds-1). The terminal also shows the installation of dependencies like libunwind8 and lm-sensors. The user is prompted to continue with the installation, and they respond with 'Y'. The terminal output confirms the successful installation of both htop and strace.

```
DebianOSProject login: user
Password:
Linux DebianOSProject 6.12.74+deb13+1-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.74+2 (2026-03-08) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
user@DebianOSProject:~$ sudo apt install htop strace
[sudo] password for user:
Installing:
  htop  strace

Installing dependencies:
  libunwind8

Suggested packages:
  lm-sensors

Summary:
  Upgrading: 0, Installing: 3, Removing: 0, Not Upgrading: 0
  Download size: 1,650 kB
  Space needed: 3,801 kB / 13.9 GB available

Continue? [Y/n]
Get:1 http://deb.debian.org/debian trixie/main amd64 htop amd64 3.4.1-5 [171 kB]
Get:2 http://deb.debian.org/debian trixie/main amd64 libunwind8 amd64 1.8.1-0.1 [56.3 kB]
Get:3 http://deb.debian.org/debian trixie/main amd64 strace amd64 6.13+ds-1 [1,423 kB]
Fetched 1,650 kB in 6s (282 kB/s)
Selecting previously unselected package htop.
(Reading database ... 34329 files and directories currently installed.)
Preparing to unpack .../htop_3.4.1-5_amd64.deb ...
Unpacking htop (3.4.1-5) ...
Selecting previously unselected package libunwind8:amd64.
Preparing to unpack .../libunwind8_1.8.1-0.1_amd64.deb ...
Unpacking libunwind8:amd64 (1.8.1-0.1) ...
Selecting previously unselected package strace.
Preparing to unpack .../strace_6.13+ds-1_amd64.deb ...
Unpacking strace (6.13+ds-1) ...
Setting up htop (3.4.1-5) ...
Setting up libunwind8:amd64 (1.8.1-0.1) ...
Setting up strace (6.13+ds-1) ...
Processing triggers for man-db (2.13.1-1) ...
Processing triggers for libc-bin (2.41-12+deb13u2) ...
user@DebianOSProject:~$
```

Terminal showing apt install output confirming successful installation of htop and strace.

4.3.2 Tool Descriptions

top

top is a real-time, interactive process monitoring utility included in virtually all Linux distributions. It displays a dynamic view of running processes, sorted by CPU usage by default, along with system-level statistics such as load average, total tasks, memory usage, and swap utilization. Key fields displayed include PID (process identifier), USER, %CPU, %MEM, VIRT (virtual memory), RES (resident memory), and the process command.

htop

htop is an enhanced, ncurses-based alternative to top. It provides a more visually organized interface, including color-coded CPU and memory meter bars for each processor core, a tree view of parent-child process relationships, and mouse support. Unlike top, htop allows the user to scroll horizontally and vertically through the process list, send signals to processes directly from the interface, and filter processes by name. These features make it particularly well-suited for the kind of detailed analysis required in this case study.

strace

strace is a diagnostic and debugging utility that intercepts and records the system calls made by a running process, as well as the signals received by it. It operates by attaching to a target process using the ptrace() kernel interface. For each intercepted system call, strace reports the call name, its arguments, and its return value. This makes it an invaluable tool for understanding how user-space programs interact with the Linux kernel at a low level.

```
DebianOSProject [Running] - Oracle VirtualBox
File Machine View Input Devices Help

top - 18:20:11 up 2:16, 1 user, load average: 0.00, 0.00, 0.00
Tasks: 88 total, 1 running, 87 sleeping, 0 stopped, 0 zombie
%Cpu(s):  0.0 us,  0.3 sy,  0.0 ni, 99.7 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :  967.4 total,  594.8 free,  277.6 used,  228.3 buff/cache
MiB Swap:  894.0 total,  894.0 free,  0.0 used,  689.8 avail Mem

  PID USER PR NI VIRT RES SHR S %CPU %MEM    TIME+    COMMAND
  10 root  20  0  0  0  0 I  0.3  0.0  0:13.46    kworker/0:1-events
  1 root  20  0 24028 14576 10760 S  0.0  1.5  0:00.79    systemd
  2 root  20  0  0  0  0 S  0.0  0.0  0:00.00    kthreadd
  3 root  20  0  0  0  0 S  0.0  0.0  0:00.00    pool_workqueue_release
  4 root  0 -20  0  0  0 I  0.0  0.0  0:00.00    kworker/R-kvfree_rcu_reclaim
  5 root  0 -20  0  0  0 I  0.0  0.0  0:00.00    kworker/R-rcu_gp
  6 root  0 -20  0  0  0 I  0.0  0.0  0:00.00    kworker/R-sync_wq
  7 root  0 -20  0  0  0 I  0.0  0.0  0:00.00    kworker/R-slab_flushwq
  8 root  0 -20  0  0  0 I  0.0  0.0  0:00.00    kworker/R-netns
  12 root  20  0  0  0  0 I  0.0  0.0  0:00.13    kworker/u4:0-flush-8:0
  13 root  0 -20  0  0  0 I  0.0  0.0  0:00.00    kworker/R-mm_percpu_wq
  14 root  20  0  0  0  0 I  0.0  0.0  0:00.00    rcu_tasks_kthread
  15 root  20  0  0  0  0 I  0.0  0.0  0:00.00    rcu_tasks_rude_kthread
  16 root  20  0  0  0  0 I  0.0  0.0  0:00.00    rcu_tasks_trace_kthread
  17 root  20  0  0  0  0 S  0.0  0.0  0:00.05    ksoftirqd/0
  18 root  20  0  0  0  0 I  0.0  0.0  0:00.03    rcu_preempt
  19 root  20  0  0  0  0 S  0.0  0.0  0:00.00    rcu_exp_per_gp_kthread_worker/0
  20 root  20  0  0  0  0 S  0.0  0.0  0:00.00    rcu_exp_gp_kthread_worker/0
  21 root  rt  0  0  0  0 S  0.0  0.0  0:00.01    migration/0
  22 root -51  0  0  0  0 S  0.0  0.0  0:00.00    idle_inject/0
  23 root  20  0  0  0  0 S  0.0  0.0  0:00.00    cpuhp/0
  25 root  20  0  0  0  0 S  0.0  0.0  0:00.00    kdevtmpfs
  26 root  0 -20  0  0  0 I  0.0  0.0  0:00.00    kworker/R-inet_frag_wq
  27 root  20  0  0  0  0 S  0.0  0.0  0:00.00    kauditd
  28 root  20  0  0  0  0 S  0.0  0.0  0:00.00    khungtaskd
  30 root  20  0  0  0  0 S  0.0  0.0  0:00.00    oom_reaper
  32 root  0 -20  0  0  0 I  0.0  0.0  0:00.00    kworker/R-writeback
  33 root  20  0  0  0  0 S  0.0  0.0  0:00.27    kcompactd0
  34 root  25  5  0  0  0 S  0.0  0.0  0:00.00    ksmd
  35 root  39 19  0  0  0 S  0.0  0.0  0:00.03    khugepaged
  36 root  0 -20  0  0  0 I  0.0  0.0  0:00.00    kworker/R-integrityd
  37 root  0 -20  0  0  0 I  0.0  0.0  0:00.00    kworker/R-kblockd
  38 root  0 -20  0  0  0 I  0.0  0.0  0:00.00    kworker/R-bkcg_punt_bio
  39 root -51  0  0  0  0 S  0.0  0.0  0:00.00    irq/9-acpi
  40 root  0 -20  0  0  0 I  0.0  0.0  0:00.00    kworker/R-tpm_dev_wq
  41 root  0 -20  0  0  0 I  0.0  0.0  0:00.00    kworker/R-edac-poller
  42 root  0 -20  0  0  0 I  0.0  0.0  0:00.00    kworker/R-devfreq_wq
  43 root  0 -20  0  0  0 I  0.0  0.0  0:00.00    kworker/R-quota_events_unbound
  44 root  0 -20  0  0  0 I  0.0  0.0  0:00.05    kworker/0:1H-kblockd
  45 root  20  0  0  0  0 S  0.0  0.0  0:00.00    kswapd0
  52 root  0 -20  0  0  0 I  0.0  0.0  0:00.00    kworker/R-kthrotld
  56 root  0 -20  0  0  0 I  0.0  0.0  0:00.00    kworker/R-acpi_thermal_pm
  57 root  0 -20  0  0  0 I  0.0  0.0  0:00.00    kworker/R-mld
```

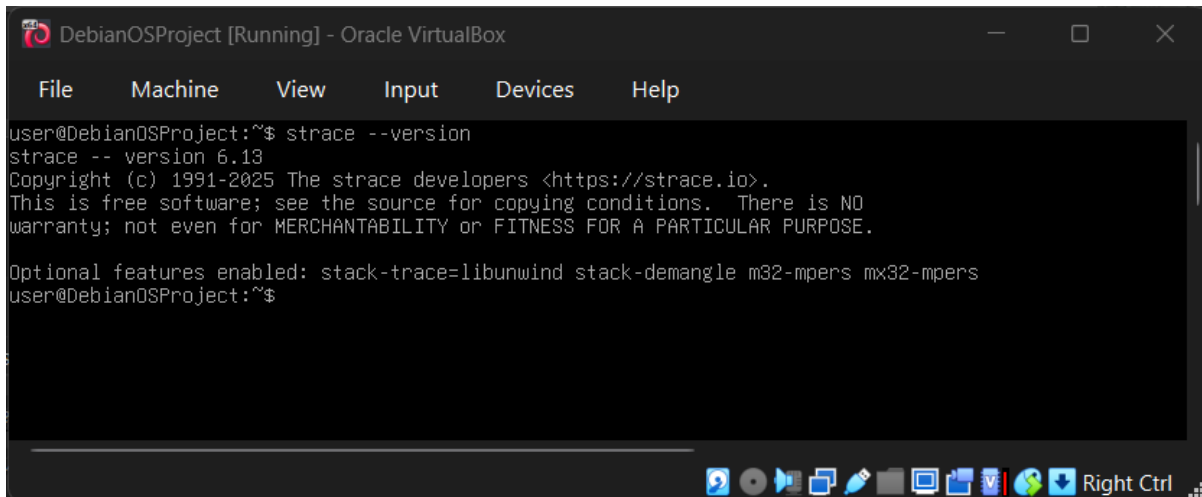
'top' running in the terminal, displaying the live process list and system resource summary.

```
DebianOSProject [Running] - Oracle VirtualBox
File Machine View Input Devices Help

CPU[ 0.0%] Tasks: 23, 1 thr, 65 kthr; 1 running
Mem[ 143M/967M] Load average: 0.00 0.00 0.00
Sup[ 0K/894M] Uptime: 01:18:13

Main T/O
  PID USER PR NI VIRT RES SHR S %CPU %MEM    TIME+    Command
  1 root  20  0 23496 14684 10748 S  0.0  1.5  0:00.71 /sbin/init
 263 root  20  0 50944 16288 15024 S  0.0  1.6  0:00.09 /usr/lib/systemd/systemd-journald
 302 system-ti 20  0 91868 8128 6992 S  0.0  0.8  0:00.04 /usr/lib/systemd/systemd-timesyncd
 315 system-ti 20  0 91868 8128  0 S  0.0  0.8  0:00.00 sd-resolve
 388 root  20  0 35264 10064 7964 S  0.0  1.0  0:00.08 /usr/lib/systemd/systemd-udevd
 575 root  20  0 6856 2904 2660 S  0.0  0.3  0:00.01 /usr/sbin/cron -f
 576 messagebus 20  0 8296 4868 4272 S  0.0  0.5  0:00.02 /usr/bin/dbus-daemon --system --address=systemd: --nofork --nopidfile --systemd-activation
 597 root  20  0 18984 3272 7988 S  0.0  0.9  0:00.05 /usr/lib/systemd/systemd-logind
 600 dhcpcd 20  0 10508 4440 3272 S  0.0  0.4  0:00.00 dhcpcd: enp0s3 [ip4] [ip6]
 601 root  20  0 10512 2632 1540 S  0.0  0.3  0:00.00 dhcpcd: [privileged proxy] enp0s3 [ip4] [ip6]
 706 dhcpcd 20  0 10512 2468 1376 S  0.0  0.2  0:00.00 dhcpcd: [DHCP6 proxy] fe80::ccc8:5aa1:8adf:2602
 708 dhcpcd 20  0 10512 2340 1248 S  0.0  0.2  0:00.00 dhcpcd: [BPF ARP] enp0s3 10.0.2.15
 727 dhcpcd 20  0 10512 2468 1376 S  0.0  0.2  0:00.00 dhcpcd: [DHCP6 proxy] fd17:625c:f037:2:6ef5:d747:63c5:e382
 761 dhcpcd 20  0 10512 2404 1312 S  0.0  0.2  0:00.00 dhcpcd: [BOOTP proxy] 10.0.2.15
 861 dhcpcd 20  0 10512 2468 1376 S  0.0  0.2  0:00.00 dhcpcd: [DHCP6 proxy] fd17:625c:f037:2:a00:27ff:fe7a:e120
 802 dhcpcd 20  0 10496 2256 1184 S  0.0  0.2  0:00.00 dhcpcd: [network proxy] enp0s3 [ip4] [ip6]
 803 dhcpcd 20  0 10488 2128 1120 S  0.0  0.2  0:00.00 dhcpcd: [control proxy] enp0s3 [ip4] [ip6]
 883 root  20  0 11768 7536 6732 S  0.0  0.8  0:00.06 sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups
 774 user  20  0 21976 12108 10012 S  0.0  1.2  0:00.06 /usr/lib/systemd/systemd --user
 776 user  20  0 24616 3832 2028 S  0.0  0.4  0:00.00 (sd-pam)
 1023 root  20  0 8156 2704 2468 S  0.0  0.3  0:00.00 /sbin/agetty -o -- \u --noproset --noclear - linux
 1025 root  20  0 9588 6116 5312 S  0.0  0.6  0:00.01 login -- user
 1036 user  20  0 8808 5712 3564 S  0.0  0.6  0:00.02 -bash
 1085 user  20  0 8192 4448 3316 R  0.7  0.4  0:00.09 htop
```

'htop' running in the terminal, showing the color-coded CPU/memory bars and process tree view.



```
user@DebianOSProject:~$ strace --version
strace -- version 6.13
Copyright (c) 1991-2025 The strace developers <https://strace.io>.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

Optional features enabled: stack-trace=libunwind stack-demangle m32-mpers mx32-mpers
user@DebianOSProject:~$
```

'strace --version' output confirming the installed version of strace.

4.4 Process and Thread Analysis

Overview

This section presents observations of Debian's process and thread management using `top`, `htop`, and related command-line utilities. The Linux kernel implements processes as individual instances of running programs, each assigned a unique Process Identifier (PID). Threads, which are sometimes referred to as lightweight processes in the Linux kernel, share the address space and resources of their parent process but are independently scheduled. In Linux, both processes and threads are internally represented by the same kernel structure, the `task_struct`, and are managed through the same scheduling infrastructure.

4.4.1 Observing Running Processes and Threads

The `'ps aux'` command was used to list all currently running processes on the system, providing a snapshot of the process table at a specific moment in time:

ps aux

To view threads alongside their parent processes, the following command was used:

ps -elf

This displays each thread as a separate line, with the LWP (Light Weight Process) column indicating the thread's own identifier and the PID column showing the process it belongs to.

DebianOSProject [Running] - Oracle VirtualBox										
File	Machine	View		Input		Devices	Help			
USER	PID	%CPU	%MEM	VSZ	RSS	TTY	STAT	START	TIME	COMMAND
root	1	0.0	1.4	23496	14684	?	Ss	18:22	0:00	/sbin/init
root	2	0.0	0.0	0	0	?	S	18:22	0:00	[kthreadd]
root	3	0.0	0.0	0	0	?	S	18:22	0:00	[pool_workqueue_release]
root	4	0.0	0.0	0	0	?	I<	18:22	0:00	[kworker/R-kvfree_rcu_reclaim]
root	5	0.0	0.0	0	0	?	I<	18:22	0:00	[kworker/R-rcu_gp]
root	6	0.0	0.0	0	0	?	I<	18:22	0:00	[kworker/R-sync_wq]
root	7	0.0	0.0	0	0	?	I<	18:22	0:00	[kworker/R-slab_flushwq]
root	8	0.0	0.0	0	0	?	I<	18:22	0:00	[kworker/R-netns]
root	10	0.2	0.0	0	0	?	I	18:22	0:11	[kworker/0:1-events]
root	12	0.0	0.0	0	0	?	I	18:22	0:00	[kworker/u4:0-events_unbound]
root	13	0.0	0.0	0	0	?	I<	18:22	0:00	[kworker/R-mm_percpu_wq]
root	14	0.0	0.0	0	0	?	I	18:22	0:00	[rcu_tasks_kthread]
root	15	0.0	0.0	0	0	?	I	18:22	0:00	[rcu_tasks_rude_kthread]
root	16	0.0	0.0	0	0	?	I	18:22	0:00	[rcu_tasks_trace_kthread]
root	17	0.0	0.0	0	0	?	S	18:22	0:00	[ksoftirqd/0]
root	18	0.0	0.0	0	0	?	I	18:22	0:00	[rcu_preempt]
root	19	0.0	0.0	0	0	?	S	18:22	0:00	[rcu_exp_par_gp_kthread_worker/0]
root	20	0.0	0.0	0	0	?	S	18:22	0:00	[rcu_exp_gp_kthread_worker]
root	21	0.0	0.0	0	0	?	S	18:22	0:00	[migration/0]
root	22	0.0	0.0	0	0	?	S	18:22	0:00	[idle_inject/0]
root	23	0.0	0.0	0	0	?	S	18:22	0:00	[cpuhp/0]
root	25	0.0	0.0	0	0	?	S	18:22	0:00	[kdevtmpfs]
root	26	0.0	0.0	0	0	?	I<	18:22	0:00	[kworker/R-inet_frag_wq]
root	27	0.0	0.0	0	0	?	S	18:22	0:00	[kauditd]
root	28	0.0	0.0	0	0	?	S	18:22	0:00	[khungtaskd]
root	29	0.0	0.0	0	0	?	I	18:22	0:00	[kworker/u4:1-flush-0:0]
root	30	0.0	0.0	0	0	?	S	18:22	0:00	[oom_reaper]
root	32	0.0	0.0	0	0	?	I<	18:22	0:00	[kworker/R-writeback]
root	33	0.0	0.0	0	0	?	S	18:22	0:00	[kcompactd0]
root	34	0.0	0.0	0	0	?	SN	18:22	0:00	[ksmd]
root	35	0.0	0.0	0	0	?	SN	18:22	0:00	[khugepaged]
root	36	0.0	0.0	0	0	?	I<	18:22	0:00	[kworker/R-kintegrityd]
root	37	0.0	0.0	0	0	?	I<	18:22	0:00	[kworker/R-kblockd]
root	38	0.0	0.0	0	0	?	I<	18:22	0:00	[kworker/R-blkcg_punt_bio]
root	39	0.0	0.0	0	0	?	S	18:22	0:00	[irq/9-acpi]
root	40	0.0	0.0	0	0	?	I<	18:22	0:00	[kworker/R-tpm_dev_wq]
root	41	0.0	0.0	0	0	?	I<	18:22	0:00	[kworker/R-edac-poller]
root	42	0.0	0.0	0	0	?	I<	18:22	0:00	[kworker/R-devfreq_wq]
root	43	0.0	0.0	0	0	?	I<	18:22	0:00	[kworker/R-quota_events_unbound]
root	44	0.0	0.0	0	0	?	I<	18:22	0:00	[kworker/0:1H-kblockd]
root	45	0.0	0.0	0	0	?	S	18:22	0:00	[kswapd0]
root	53	0.0	0.0	0	0	?	I<	18:22	0:00	[kworker/R-kthrotld]
root	57	0.0	0.0	0	0	?	I<	18:22	0:00	[kworker/R-acpi_thermal_pm]
root	58	0.0	0.0	0	0	?	I<	18:22	0:00	[kworker/R-mld]
root	59	0.0	0.0	0	0	?	I<	18:22	0:00	[kworker/R-ipv6_addrconf]
root	64	0.0	0.0	0	0	?	I<	18:22	0:00	[kworker/R-kstrp]
root	66	0.0	0.0	0	0	?	I<	18:22	0:00	[kworker/u5:0]
root	173	0.0	0.0	0	0	?	I<	18:22	0:00	[kworker/R-ata_sff]

```

DebianOSProject [Running] - Oracle VirtualBox
File Machine View Input Devices Help
root 45 0.0 0.0 0 0 ? S 18:22 0:00 [kswapd0]
root 53 0.0 0.0 0 0 ? I< 18:22 0:00 [kworker/R-kthrotld]
root 57 0.0 0.0 0 0 ? I< 18:22 0:00 [kworker/R-acpi_thermal_pm]
root 58 0.0 0.0 0 0 ? I< 18:22 0:00 [kworker/R-mld]
root 59 0.0 0.0 0 0 ? I< 18:22 0:00 [kworker/R-ipv6_addrconf]
root 64 0.0 0.0 0 0 ? I< 18:22 0:00 [kworker/R-kstrp]
root 66 0.0 0.0 0 0 ? I< 18:22 0:00 [kworker/u5:0]
root 173 0.0 0.0 0 0 ? I< 18:22 0:00 [kworker/R-ata_sff]
root 174 0.0 0.0 0 0 ? S 18:22 0:00 [scsi_ah_0]
root 175 0.0 0.0 0 0 ? I< 18:22 0:00 [kworker/R-scsi_tmf_0]
root 176 0.0 0.0 0 0 ? S 18:22 0:00 [scsi_ah_1]
root 177 0.0 0.0 0 0 ? I< 18:22 0:00 [kworker/R-scsi_tmf_1]
root 178 0.0 0.0 0 0 ? I 18:22 0:04 [kworker/0:3-ata_sff]
root 179 0.0 0.0 0 0 ? S 18:22 0:00 [scsi_ah_2]
root 180 0.0 0.0 0 0 ? I< 18:22 0:00 [kworker/R-scsi_tmf_2]
root 182 0.0 0.0 0 0 ? I< 18:22 0:00 [kworker/0:2H-kblockd]
root 213 0.0 0.0 0 0 ? S 18:22 0:00 [jbd2/sda1-8]
root 214 0.0 0.0 0 0 ? I< 18:22 0:00 [kworker/R-ext4-rsv-conversion]
root 263 0.0 1.6 50944 16288 ? Ss 18:22 0:00 /usr/lib/systemd/systemd-journald
systemd+ 302 0.0 0.8 91868 8128 ? Ssl 18:22 0:00 /usr/lib/systemd/systemd-timesyncd
root 308 0.0 1.0 35264 10064 ? Ss 18:22 0:00 /usr/lib/systemd/systemd-udev
root 316 0.0 0.0 0 0 ? S 18:22 0:00 [psimon]
root 345 0.0 0.0 0 0 ? I< 18:22 0:00 [kworker/R-cryptd]
root 380 0.0 0.0 0 0 ? I 18:22 0:00 [kworker/u4:6-kvfree_rcu_reclaim]
root 446 0.0 0.0 0 0 ? S 18:22 0:00 [irq/18-vmwgfx]
root 447 0.0 0.0 0 0 ? I< 18:22 0:00 [kworker/R-ttm]
root 575 0.0 0.2 6856 2904 ? Ss 18:22 0:00 /usr/sbin/cron -f
message+ 576 0.0 0.4 8296 4868 ? Ss 18:22 0:00 /usr/bin/dbus-daemon --system --address=systemd: --nofork --nopidfile
syslog-only
root 597 0.0 0.9 18984 9272 ? Ss 18:22 0:00 /usr/lib/systemd/systemd-logind
dhcpcd 600 0.0 0.4 10508 4440 ? S 18:22 0:00 dhcpcd: enp0s3 [ip4] [ip6]
root 601 0.0 0.2 10512 2632 ? S 18:22 0:00 dhcpcd: [privileged proxy] enp0s3 [ip4] [ip6]
dhcpcd 602 0.0 0.2 10496 2256 ? S 18:22 0:00 dhcpcd: [network proxy] enp0s3 [ip4] [ip6]
dhcpcd 603 0.0 0.2 10488 2128 ? S 18:22 0:00 dhcpcd: [control proxy] enp0s3 [ip4] [ip6]
root 683 0.0 0.8 11768 7936 ? Ss 18:22 0:00 sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups
root 696 0.0 0.0 0 0 ? I< 18:22 0:00 [kworker/R-cfg80211]
dhcpcd 706 0.0 0.2 10512 2468 ? S 18:22 0:00 dhcpcd: [DHCP6 proxy] fe80::ccc8:5aa1:8adf:2602
dhcpcd 708 0.0 0.2 10512 2340 ? S 18:22 0:00 dhcpcd: [BPF ARP] enp0s3 10.0.2.15
dhcpcd 727 0.0 0.2 10512 2468 ? S 18:22 0:00 dhcpcd: [DHCP6 proxy] fd17:625c:f037:2:6ef5:d747:63c5:e382
dhcpcd 761 0.0 0.2 10512 2404 ? S 18:22 0:00 dhcpcd: [B00TP proxy] 10.0.2.15
root 772 0.0 0.0 0 0 ? S 18:23 0:00 [psimon]
user 774 0.0 1.2 21976 12108 ? Ss 18:23 0:00 /usr/lib/systemd/systemd --user
user 776 0.0 0.3 24616 3832 ? S 18:23 0:00 (sd-pam)
dhcpcd 861 0.0 0.2 10512 2468 ? S 18:26 0:00 dhcpcd: [DHCP6 proxy] fd17:625c:f037:2:a00:27ff:fe7a:e120
root 1923 0.0 0.2 8156 2704 tty6 Ss+ 19:33 0:00 /sbin/agetty -o -- \u --noret --noclear - linux
root 1925 0.0 0.6 9588 6116 ? Ss 19:33 0:00 login -- user
user 1936 0.0 0.6 8940 5968 tty1 Ss 19:33 0:00 -bash
root 1102 0.0 0.0 0 0 ? I 19:43 0:00 [kworker/0:0]
user 1113 0.0 0.4 9384 4188 tty1 R+ 19:46 0:00 ps aux

```

Output of `ps aux` showing all running processes, their PIDs, CPU and memory usage, and command names.

DebianOSProject [Running] - Oracle VirtualBox

File	Machine	View	Input	Devices	Help				
UID	PID	PPID	LWP	C	NLWP	STIME	TTY	TIME	CMD
root	1	0	1	0	1	18:22	?	00:00:00	/sbin/init
root	2	0	2	0	1	18:22	?	00:00:00	[kthreadd]
root	3	2	3	0	1	18:22	?	00:00:00	[pool_workqueue_release]
root	4	2	4	0	1	18:22	?	00:00:00	[kworker/R-kvfree_rcu_reclaim]
root	5	2	5	0	1	18:22	?	00:00:00	[kworker/R-rcu_gp]
root	6	2	6	0	1	18:22	?	00:00:00	[kworker/R-sync_wq]
root	7	2	7	0	1	18:22	?	00:00:00	[kworker/R-slab_flushwq]
root	8	2	8	0	1	18:22	?	00:00:00	[kworker/R-netns]
root	10	2	10	0	1	18:22	?	00:00:11	[kworker/0:1-events]
root	12	2	12	0	1	18:22	?	00:00:00	[kworker/u4:0-events_unbound]
root	13	2	13	0	1	18:22	?	00:00:00	[kworker/R-mm_percpu_wq]
root	14	2	14	0	1	18:22	?	00:00:00	[rcu_tasks_kthread]
root	15	2	15	0	1	18:22	?	00:00:00	[rcu_tasks_rude_kthread]
root	16	2	16	0	1	18:22	?	00:00:00	[rcu_tasks_trace_kthread]
root	17	2	17	0	1	18:22	?	00:00:00	[ksoftirqd/0]
root	18	2	18	0	1	18:22	?	00:00:00	[rcu_preempt]
root	19	2	19	0	1	18:22	?	00:00:00	[rcu_exp_par_gp_kthread_worker/0]
root	20	2	20	0	1	18:22	?	00:00:00	[rcu_exp_gp_kthread_worker]
root	21	2	21	0	1	18:22	?	00:00:00	[migration/0]
root	22	2	22	0	1	18:22	?	00:00:00	[idle_inject/0]
root	23	2	23	0	1	18:22	?	00:00:00	[cpuhp/0]
root	25	2	25	0	1	18:22	?	00:00:00	[kdevtmpfs]
root	26	2	26	0	1	18:22	?	00:00:00	[kworker/R-inet_frag_wq]
root	27	2	27	0	1	18:22	?	00:00:00	[kauditd]
root	28	2	28	0	1	18:22	?	00:00:00	[khungtaskd]
root	29	2	29	0	1	18:22	?	00:00:00	[kworker/u4:1-flush-8:0]
root	30	2	30	0	1	18:22	?	00:00:00	[oom_reaper]
root	32	2	32	0	1	18:22	?	00:00:00	[kworker/R-writeback]
root	33	2	33	0	1	18:22	?	00:00:00	[kcompactd0]
root	34	2	34	0	1	18:22	?	00:00:00	[ksmd]
root	35	2	35	0	1	18:22	?	00:00:00	[khugepaged]
root	36	2	36	0	1	18:22	?	00:00:00	[kworker/R-kintegrityd]
root	37	2	37	0	1	18:22	?	00:00:00	[kworker/R-kblockd]
root	38	2	38	0	1	18:22	?	00:00:00	[kworker/R-blkg_punt_bio]
root	39	2	39	0	1	18:22	?	00:00:00	[irq/9-acpi]
root	40	2	40	0	1	18:22	?	00:00:00	[kworker/R-tpm_dev_wq]
root	41	2	41	0	1	18:22	?	00:00:00	[kworker/R-edac-poller]
root	42	2	42	0	1	18:22	?	00:00:00	[kworker/R-devfreq_wq]
root	43	2	43	0	1	18:22	?	00:00:00	[kworker/R-quota_events_unbound]
root	44	2	44	0	1	18:22	?	00:00:00	[kworker/0:1H-kblockd]
root	45	2	45	0	1	18:22	?	00:00:00	[kswapd0]
root	53	2	53	0	1	18:22	?	00:00:00	[kworker/R-kthrotld]
root	57	2	57	0	1	18:22	?	00:00:00	[kworker/R-acpi_thermal_pm]
root	58	2	58	0	1	18:22	?	00:00:00	[kworker/R-mld]
root	59	2	59	0	1	18:22	?	00:00:00	[kworker/R-ipv6_addrconf]
root	64	2	64	0	1	18:22	?	00:00:00	[kworker/R-kstrp]
root	66	2	66	0	1	18:22	?	00:00:00	[kworker/u5:0]
root	173	2	173	0	1	18:22	?	00:00:00	[kworker/R-ata_sff]

"psAuxOut" 90 lines, 7992 bytes

File	Machine	View	Input	Devices	Help
root	53	2	53 0	1 18:22 ?	00:00:00 [kworker/R-kthrotid]
root	57	2	57 0	1 18:22 ?	00:00:00 [kworker/R-acpi_thermal_pm]
root	58	2	58 0	1 18:22 ?	00:00:00 [kworker/R-mld]
root	59	2	59 0	1 18:22 ?	00:00:00 [kworker/R-ipv6_addrconf]
root	64	2	64 0	1 18:22 ?	00:00:00 [kworker/R-kstrp]
root	66	2	66 0	1 18:22 ?	00:00:00 [kworker/u6:0]
root	173	2	173 0	1 18:22 ?	00:00:00 [kworker/R-ata_sff]
root	174	2	174 0	1 18:22 ?	00:00:00 [scsi_ah_0]
root	175	2	175 0	1 18:22 ?	00:00:00 [kworker/R-scsi_tmf_0]
root	176	2	176 0	1 18:22 ?	00:00:00 [scsi_ah_1]
root	177	2	177 0	1 18:22 ?	00:00:00 [kworker/R-scsi_tmf_1]
root	178	2	178 0	1 18:22 ?	00:00:04 [kworker/0:3-events]
root	179	2	179 0	1 18:22 ?	00:00:00 [scsi_ah_2]
root	180	2	180 0	1 18:22 ?	00:00:00 [kworker/R-scsi_tmf_2]
root	182	2	182 0	1 18:22 ?	00:00:00 [kworker/0:2H-kblockd]
root	213	2	213 0	1 18:22 ?	00:00:00 [jbd2/sda1-8]
root	214	2	214 0	1 18:22 ?	00:00:00 [kworker/R-ext4-rsv-conversion]
root	263	1	263 0	1 18:22 ?	00:00:00 /usr/lib/systemd/systemd-journald
systemd+	302	1	302 0	2 18:22 ?	00:00:00 /usr/lib/systemd/systemd-timesyncd
systemd+	302	1	315 0	2 18:22 ?	00:00:00 /usr/lib/systemd/systemd-timesyncd
root	308	1	308 0	1 18:22 ?	00:00:00 /usr/lib/systemd/systemd-udev
root	316	2	316 0	1 18:22 ?	00:00:00 [psimon]
root	345	2	345 0	1 18:22 ?	00:00:00 [kworker/R-cryptd]
root	380	2	380 0	1 18:22 ?	00:00:00 [kworker/u4:6-ipv6_addrconf]
root	446	2	446 0	1 18:22 ?	00:00:00 [irq/18-vmwgfx]
root	447	2	447 0	1 18:22 ?	00:00:00 [kworker/R-ttm]
root	575	1	575 0	1 18:22 ?	00:00:00 /usr/sbin/cron -f
message+	576	1	576 0	1 18:22 ?	00:00:00 /usr/bin/dbus-daemon --system --address=systemd: --nofork --nopidfile --systemd-activation --su
slog-only					
root	597	1	597 0	1 18:22 ?	00:00:00 /usr/lib/systemd/systemd-logind
dhcpcd	600	1	600 0	1 18:22 ?	00:00:00 dhcpcd: enp0s3 [ip4] [ip6]
root	601	600	601 0	1 18:22 ?	00:00:00 dhcpcd: [privileged proxy] enp0s3 [ip4] [ip6]
dhcpcd	602	600	602 0	1 18:22 ?	00:00:00 dhcpcd: [network proxy] enp0s3 [ip4] [ip6]
dhcpcd	603	600	603 0	1 18:22 ?	00:00:00 dhcpcd: [control proxy] enp0s3 [ip4] [ip6]
root	683	1	683 0	1 18:22 ?	00:00:00 sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups
root	696	2	696 0	1 18:22 ?	00:00:00 [kworker/R-cfg80211]
dhcpcd	706	601	706 0	1 18:22 ?	00:00:00 dhcpcd: [DHCP6 proxy] fe80::ccc8:5aa1:8adf:2602
dhcpcd	708	601	708 0	1 18:22 ?	00:00:00 dhcpcd: [BPF ARP] enp0s3 10.0.2.15
dhcpcd	727	601	727 0	1 18:22 ?	00:00:00 dhcpcd: [DHCP6 proxy] fd17:625c:f037:2:6ef5:d747:63c5:e382
dhcpcd	761	601	761 0	1 18:22 ?	00:00:00 dhcpcd: [BOOTP proxy] 10.0.2.15
root	772	2	772 0	1 18:23 ?	00:00:00 [psimon]
user	774	1	774 0	1 18:23 ?	00:00:00 /usr/lib/systemd/systemd --user
user	776	774	776 0	1 18:23 ?	00:00:00 (sd-pam)
dhcpcd	861	601	861 0	1 18:26 ?	00:00:00 dhcpcd: [DHCP6 proxy] fd17:625c:f037:2:a00:27ff:fe7a:e120
root	1023	1	1023 0	1 19:33 tty6	00:00:00 /sbin/agetty -o - -u --noreset --noclear - linux
root	1025	1	1025 0	1 19:33 ?	00:00:00 login -- user
user	1036	1025	1036 0	1 19:33 tty1	00:00:00 -bash
root	1124	2	1124 0	1 19:48 ?	00:00:00 [kworker/0:0]
user	1145	1036	1145 0	1 19:49 tty1	00:00:00 ps -eLf

Output of ps -eLf showing processes alongside their associated threads (LWP column).

4.4.2 Identifying Parent–Child Relationships

In Linux, every process except the initial process (PID 1, typically systemd in Debian) has a parent process. When a process is created via the fork() system call, it inherits many attributes of its parent and is linked to it through the PPID (Parent Process Identifier) field in the kernel's process table. htop was used in tree mode (activated with F5) to display the process hierarchy interactively, clearly showing which processes were spawned by systemd and which were spawned by the user's shell session.

htop in tree view (F5 mode) showing parent–child process relationships in a hierarchical list.

4.4.3 Monitoring CPU and Memory Usage

CPU and memory consumption were monitored in real time using htop. The top section of the htop display shows individual bars for each CPU core, with colors distinguishing between user-space CPU time (green), kernel/system CPU time (red), and I/O wait (blue). The memory bar similarly distinguishes between used memory, buffers, and cache.

htop display showing CPU and memory bars at the top, with several processes consuming measurable memory resources.

4.4.4 Demonstrating Process Creation and Termination

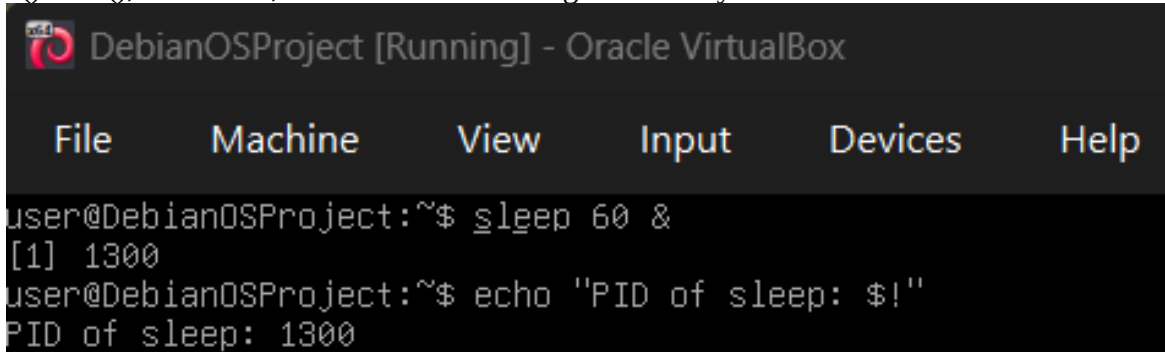
To observe process creation in real time, a background process was deliberately spawned from the terminal using sleep:

```
sleep 60 &
echo "PID of sleep: $!"
```

The newly created process was then observed in htop and terminated using the kill command:

kill [PID]

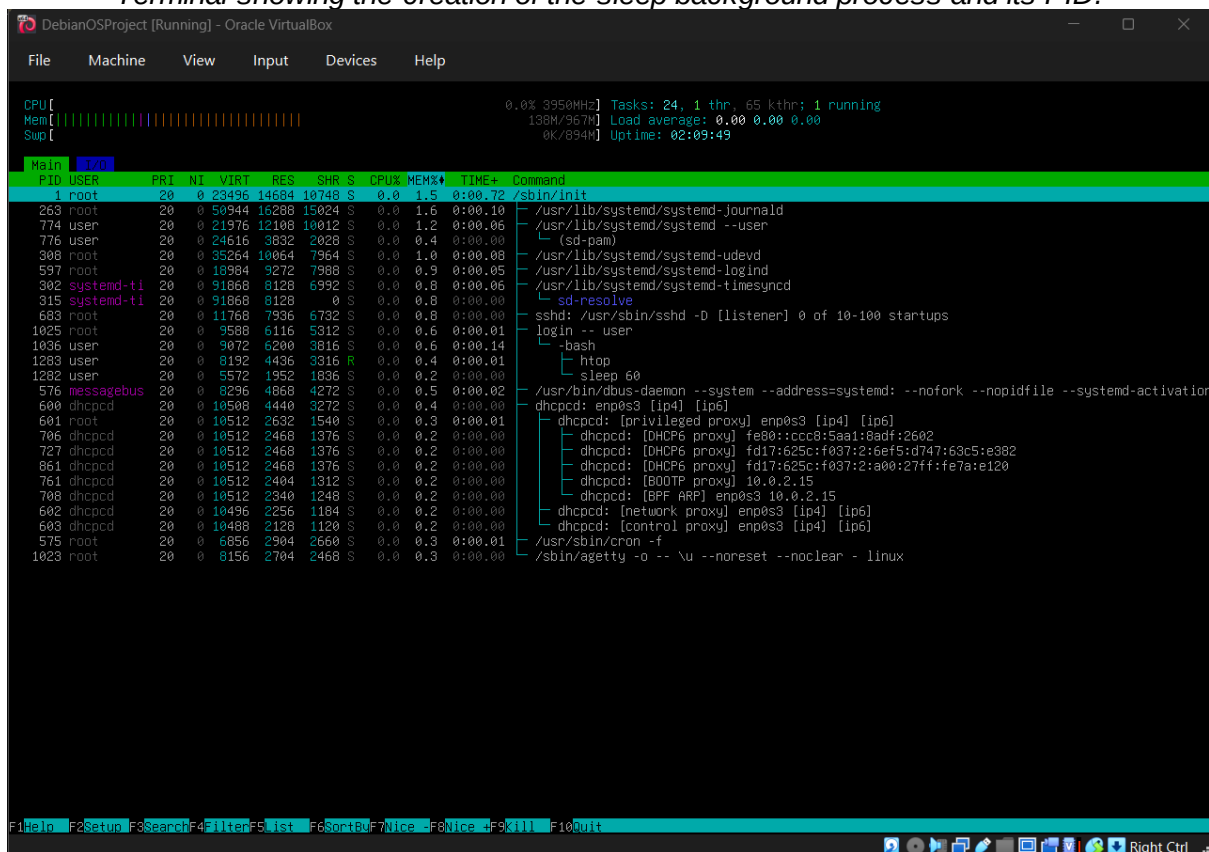
This sequence demonstrates the full lifecycle of a Linux process: creation via fork()/exec(), execution, and termination via signal delivery.



```
DebianOSProject [Running] - Oracle VirtualBox
File Machine View Input Devices Help

user@DebianOSProject:~$ sleep 60 &
[1] 1300
user@DebianOSProject:~$ echo "PID of sleep: $!"
PID of sleep: 1300
```

Terminal showing the creation of the sleep background process and its PID.



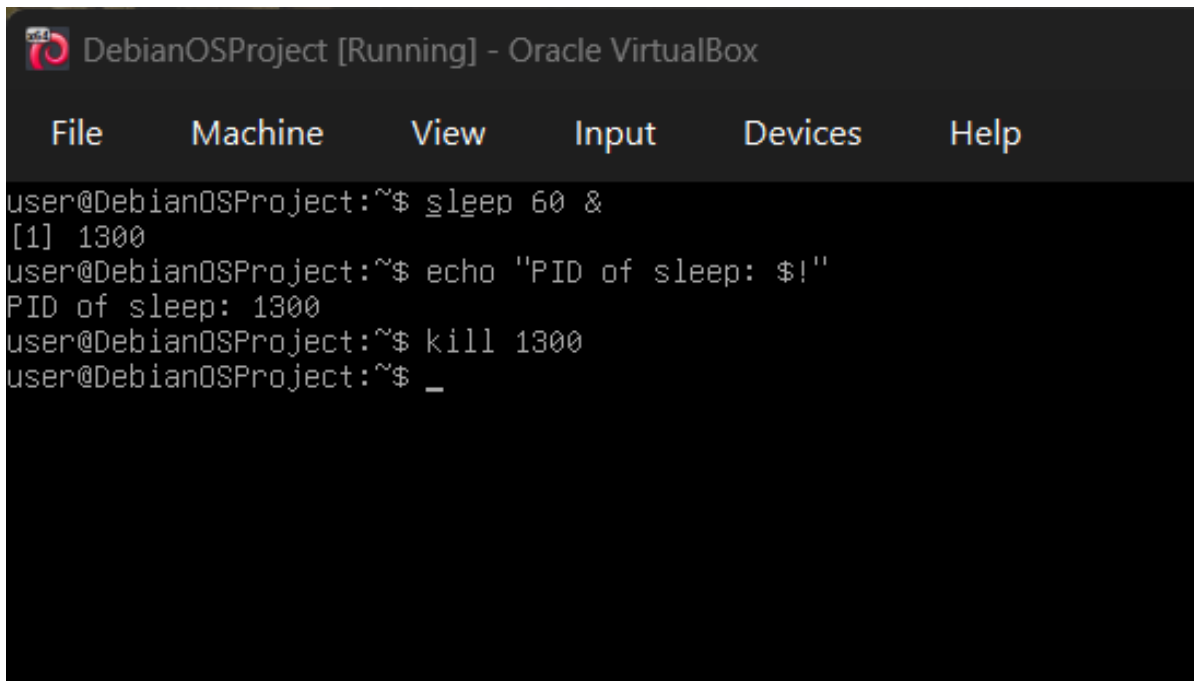
```
DebianOSProject [Running] - Oracle VirtualBox
File Machine View Input Devices Help

CPU[ 0.0% 3950MHz] Tasks: 24, 1 thr, 65 kthr, 1 running
Mem[ 138M/957M] Load average: 0.00 0.00 0.00
Sup[ 0K/894K] Uptime: 02:09:49

Main 57%
PID USER PRI NI VIRT RES SHR S CPU% MEM% TIME+ Command
1 root 20 0 23496 14684 10748 S 0.0 1.5 0:00.72 /sbin/init
263 root 20 0 50944 16288 15024 S 0.0 1.6 0:00.10 /usr/lib/systemd/systemd-journald
774 user 20 0 21976 12108 10012 S 0.0 1.2 0:00.06 /usr/lib/systemd/systemd --user
776 user 20 0 24616 3832 2028 S 0.0 0.4 0:00.00 (sd-pam)
308 root 20 0 35264 10064 7964 S 0.0 1.0 0:00.08 /usr/lib/systemd/systemd-udevd
597 root 20 0 18984 9272 7908 S 0.0 0.9 0:00.05 /usr/lib/systemd/systemd-logind
302 systemd-tl 20 0 91868 8128 6992 S 0.0 0.8 0:00.06 /usr/lib/systemd/systemd-timesyncd
315 systemd-tl 20 0 91868 8128 0 S 0.0 0.8 0:00.00 sd-resolve
683 root 20 0 11768 7936 6732 S 0.0 0.8 0:00.00 sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups
1025 root 20 0 9588 6116 5312 S 0.0 0.6 0:00.01 login -- user
1036 user 20 0 9072 6200 3816 S 0.0 0.6 0:00.14 -bash
1283 user 20 0 8192 4436 3316 R 0.0 0.4 0:00.01 htop
1282 user 20 0 5572 1952 1836 S 0.0 0.2 0:00.00 sleep 60
576 messagebus 20 0 8236 4868 4272 S 0.0 0.5 0:00.02 /usr/bin/dbus-daemon --system --address=systemd: --nofork --nopidfile --systemd-activation
600 dhcpcd 20 0 10508 4440 3272 S 0.0 0.4 0:00.00 dhcpcd: enp0s3 [ip4] [ip6]
601 root 20 0 13512 2632 1540 S 0.0 0.3 0:00.01 dhcpcd: [privileged proxy] enp0s3 [ip4] [ip6]
706 dhcpcd 20 0 13512 2468 1376 S 0.0 0.2 0:00.00 dhcpcd: [DHCP6 proxy] fe80::ccc8:5aa1:8adf:2602
727 dhcpcd 20 0 13512 2468 1376 S 0.0 0.2 0:00.00 dhcpcd: [DHCP6 proxy] fd17:625c:f037:2:6ef5:d747:63c5:e382
861 dhcpcd 20 0 13512 2468 1376 S 0.0 0.2 0:00.00 dhcpcd: [DHCP6 proxy] fd17:625c:f037:2:a00:27ff:fe7a:e120
761 dhcpcd 20 0 13512 2404 1312 S 0.0 0.2 0:00.00 dhcpcd: [BDOTP proxy] 10.0.2.15
708 dhcpcd 20 0 13512 2340 1248 S 0.0 0.2 0:00.00 dhcpcd: [BPF ARP] enp0s3 10.0.2.15
602 dhcpcd 20 0 10496 2256 1184 S 0.0 0.2 0:00.00 dhcpcd: [network proxy] enp0s3 [ip4] [ip6]
603 dhcpcd 20 0 10488 2128 1120 S 0.0 0.2 0:00.00 dhcpcd: [control proxy] enp0s3 [ip4] [ip6]
575 root 20 0 6856 2904 2660 S 0.0 0.3 0:00.01 /usr/sbin/cron -f
1023 root 20 0 8156 2704 2468 S 0.0 0.3 0:00.00 /sbin/agetty -o -- \u --noproset --noclear - linux

File Edit F2Setup F3Search F4Filter F5List F6SortBy F7Nice F8Nice F9Kill F10Quit
```

Caption: htop capturing the sleep process in the process list before termination.



```
DebianOSProject [Running] - Oracle VirtualBox
File Machine View Input Devices Help
user@DebianOSProject:~$ sleep 60 &
[1] 1300
user@DebianOSProject:~$ echo "PID of sleep: $!"
PID of sleep: 1300
user@DebianOSProject:~$ kill 1300
user@DebianOSProject:~$ _
```

Terminal confirming process termination after the kill command was issued.

4.5 System Call Tracing

Overview

System calls are the fundamental interface between user-space programs and the Linux kernel. Whenever a process needs to perform a privileged operation such as reading a file, creating a new process, or allocating memory, it must invoke a system call. On Debian (x86-64), this is done via the syscall instruction, which transitions the processor from user mode (Ring 3) to kernel mode (Ring 0), where the requested operation is carried out. strace was used to trace and document these interactions for several representative programs.

4.5.1 Tracing System Calls Related to Process Creation

The fork() and execve() system calls are central to process creation in Linux. fork() creates a new process by duplicating the calling process, and execve() replaces the current process image with a new program. To observe these calls in meaningful depth, strace was used to trace a shell pipeline rather than a single command. A plain program such as ls executes too quickly and involves no child process creation of its own, producing only a single execve() call and little else. A shell pipeline, by contrast, forces bash to call clone() to fork a child, execve() to load the target program into that child, and wait4() to reap the child upon completion — demonstrating the full process creation lifecycle in a single trace.

The following command was used, with the -f flag instructing strace to follow into child processes and prefix each line with its PID:

```
strace -f -e trace=execve,clone,wait4 bash -c "ls | wc -l" 2>&1 | head -40
```

The pipeline ls | wc -l spawns two child processes — one for ls and one for wc — making the output particularly informative: multiple clone() calls appear (one per child), each followed by an execve() in the respective child process, and wait4() calls in the parent as it waits for both children to exit. Each line in the output is prefixed with the PID of the process that made the call, making it straightforward to distinguish parent from child activity.

The `clone()` system call is of particular interest on modern Linux: rather than the older `fork()`, the C library typically invokes `clone()` with a set of flags that precisely control which resources are shared between parent and child. When creating a new process, flags such as `SIGCHLD` are passed; when creating a thread, flags like `CLONE_VM` | `CLONE_FS` | `CLONE_FILES` | `CLONE_SIGHAND` are added so that the new thread shares the address space and file descriptors of its parent. This makes `clone()` the single unified kernel mechanism underlying both process and thread creation on Linux.

```

DebianOSProject [Running] - Oracle VirtualBox
File Machine View Input Devices Help
user@DebianOSProject:~$ strace -f -e trace=execve,clone,wait4 bash -c "ls | wc -l" 2>&1 | head -40
execve("/usr/bin/bash", ["bash", "-c", "ls | wc -l"], 0x7fff427081d8 /* 20 vars */) = 0
clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD, strace: Process 1354 attached
, child_tidptr=0x7f6a59c4ee50) = 1354
[pid 1353] clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD, strace: Process 1355 attached
, child_tidptr=0x7f6a59c4ee50) = 1355
[pid 1353] wait4(-1, <unfinished ...>
[pid 1354] execve("/usr/bin/ls", ["ls"], 0x560e50547ec0 /* 20 vars */ <unfinished ...>
[pid 1355] execve("/usr/bin/wc", ["wc", "-l"], 0x560e50547ec0 /* 20 vars */ <unfinished ...>
[pid 1354] <... execve resumed>      = 0
[pid 1355] <... execve resumed>      = 0
0
[pid 1354] +++ exited with 0 +++
[pid 1353] <... wait4 resumed> [{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 1354
[pid 1353] wait4(-1, <unfinished ...>
[pid 1355] +++ exited with 0 +++
<... wait4 resumed> [{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 1355
--- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=1354, si_uid=1000, si_status=0, si_utime=0, si_stime=0} ---
wait4(-1, 0x7ffe6e730dd0, WNOHANG, NULL) = -1 ECHILD (No child processes)
+++ exited with 0 +++
user@DebianOSProject:~$ _

```

strace -f output for `bash -c "ls | wc -l"`, showing `clone()` calls from the parent bash process, followed by `execve()` in each child process (PID-prefixed), and `wait4()` as the parent waits for children to terminate.

4.5.2 Tracing System Calls Related to File Operations

File I/O operations in Linux are mediated through a set of well-defined system calls: `open()` (or `openat()`), `read()`, `write()`, `close()`, and `stat()`. To observe these, `strace` was used to trace a simple file read operation:

strace -e trace=openat,read,write,close,stat cat /etc/hostname

The output reveals the sequence of kernel interactions required to open and read even a simple file, illustrating the degree to which user-space programs depend on the kernel for access to hardware and filesystem resources.

DebianOSProject [Running] - Oracle VirtualBox

File	Machine	View	Input	Devices	Help
------	---------	------	-------	---------	------

```
user@DebianOSProject:~$ strace -c ls
```

% time	seconds	usecs/call	calls	errors	syscall
0.00	0.000000	0	6		read
0.00	0.000000	0	10		close
0.00	0.000000	0	8		fstat
0.00	0.000000	0	23		mmap
0.00	0.000000	0	6		mprotect
0.00	0.000000	0	1		munmap
0.00	0.000000	0	3		brk
0.00	0.000000	0	2		ioctl
0.00	0.000000	0	2		pread64
0.00	0.000000	0	2	2	access
0.00	0.000000	0	1		execve
0.00	0.000000	0	2	2	statfs
0.00	0.000000	0	6	4	prctl
0.00	0.000000	0	1		arch_prctl
0.00	0.000000	0	2		getdents64
0.00	0.000000	0	1		set_tid_address
0.00	0.000000	0	8		openat
0.00	0.000000	0	1		set_robust_list
0.00	0.000000	0	1		prlimit64
0.00	0.000000	0	1		getrandom
0.00	0.000000	0	1		rseq
100.00	0.000000	0	88	8	total

```
user@DebianOSProject:~$ _
```

strace -c ls summary output showing a tabulated count and time breakdown of all system calls made by ls.

4.6 Stress Testing and Performance Evaluation

Overview

Stress testing was performed to observe how Debian's kernel manages resource contention under high load. The goal was to push CPU, memory, and I/O subsystems toward their limits in a controlled manner and to document the resulting changes in process scheduling, memory management, and overall system responsiveness. The tools stress and stress-ng were used for this purpose.

4.6.1 Installing Stress Testing Tools

```
sudo apt install stress stress-ng -y
```

```

DebianOSProject [Running] - Oracle VirtualBox

File Machine View Input Devices Help

user@DebianOSProject:~$ sudo apt install stress stress-ng -y
stress is already the newest version (1.0.7-1).
stress-ng is already the newest version (0.19.02-1).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 0

```

Terminal output confirming successful installation of stress and stress-ng.

4.6.2 CPU Stress Test

A CPU stress test was initiated to saturate all available virtual CPU cores:

stress --cpu 1 --timeout 30s

Where 1 corresponds to the number of virtual CPU cores allocated to the VM. During this test, htop was observed in a separate terminal to document the impact on the CPU usage bars, scheduler behavior, and load average.

```

DebianOSProject [Running] - Oracle VirtualBox

File Machine View Input Devices Help

CPU[|||||] 100.0%[3950MHz] Tasks: 25, 1 thr, 65 kthr; 1 running
Mem[|||||] 110M/967M Load average: 0.34 0.10 0.04
Swap[|||||] 0K/894K Uptime: 02:42:06

Main 170

PID USER PRI NI VIRT RES SHR S CPUX MEMS% TIME+ Command
1 root 20 0 23496 14684 10748 S 0.0 1.5 0:00.73 /sbin/init
263 root 20 0 50944 16352 15088 S 0.0 1.7 0:00.10 /usr/lib/systemd/systemd-journald
774 user 20 0 21976 12108 10012 S 0.0 1.2 0:00.06 /usr/lib/systemd/systemd --user
776 user 20 0 24616 3832 2028 S 0.0 0.4 0:00.00 (sd-pam)
308 root 20 0 35264 10064 7964 S 0.0 1.0 0:00.08 /usr/lib/systemd/systemd-udevd
597 root 20 0 18984 9272 7988 S 0.0 0.9 0:00.06 /usr/lib/systemd/systemd-logind
302 systemd-ti 20 0 91868 8128 6992 S 0.0 0.8 0:00.06 /usr/lib/systemd/systemd-timesyncd
315 systemd-ti 20 0 91868 8128 0 S 0.0 0.8 0:00.00 sd-resolve
683 root 20 0 11768 7936 6732 S 0.0 0.8 0:00.00 sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups
1025 root 20 0 9588 6116 5312 S 0.0 0.6 0:00.01 login -- user
1836 user 20 0 9216 6384 3816 S 0.0 0.6 0:00.23 -bash
1814 user 20 0 6360 4712 3480 R 0.0 0.5 0:00.01 htop
1812 user 20 0 3524 2032 1980 S 0.0 0.2 0:00.00 stress --cpu 1 --timeout 30s
1813 user 20 0 3524 568 496 R 100.0 0.1 0:05.89 stress --cpu 1 --timeout 30s
576 messagebus 20 0 8296 4872 4272 S 0.0 0.5 0:00.02 /usr/bin/dbus-daemon --system --address=systemd: --nofork --nopidfile --systemd-activation
600 dhcpcd 20 0 10508 4440 3272 S 0.0 0.4 0:00.00 dhcpcd: enp0s3 [ip4] [ip6]
601 root 20 0 10512 2632 1540 S 0.0 0.3 0:00.01 dhcpcd: [privileged proxy] enp0s3 [ip4] [ip6]
706 dhcpcd 20 0 10512 2468 1376 S 0.0 0.2 0:00.00 dhcpcd: [DHCP6 proxy] fe80::cccc8:5aa1:8adf:2602
727 dhcpcd 20 0 10512 2468 1376 S 0.0 0.2 0:00.00 dhcpcd: [DHCP6 proxy] fd17:625c:f037:2:6ef5:d747:63c5:e982
861 dhcpcd 20 0 10512 2468 1376 S 0.0 0.2 0:00.00 dhcpcd: [DHCP6 proxy] fd17:625c:f037:2:a00:27ff:fe7a:e120
761 dhcpcd 20 0 10512 2404 1312 S 0.0 0.2 0:00.00 dhcpcd: [BOOTP proxy] 10.0.2.15
708 dhcpcd 20 0 10512 2340 1248 S 0.0 0.2 0:00.00 dhcpcd: [BPF ARP] enp0s3 10.0.2.15
602 dhcpcd 20 0 10496 2256 1184 S 0.0 0.2 0:00.00 dhcpcd: [network proxy] enp0s3 [ip4] [ip6]
603 dhcpcd 20 0 10488 2128 1120 S 0.0 0.2 0:00.00 dhcpcd: [control proxy] enp0s3 [ip4] [ip6]
575 root 20 0 6856 2904 2660 S 0.0 0.3 0:00.01 /usr/sbin/cron -f
1023 root 20 0 8156 2704 2468 S 0.0 0.3 0:00.00 /sbin/agetty -o - - \u --noneset --noclear - linux

F1Help F2Setup F3Search F4Filter F5List F6SortBy F7Nice F8Nice F9Kill F10Quit

```

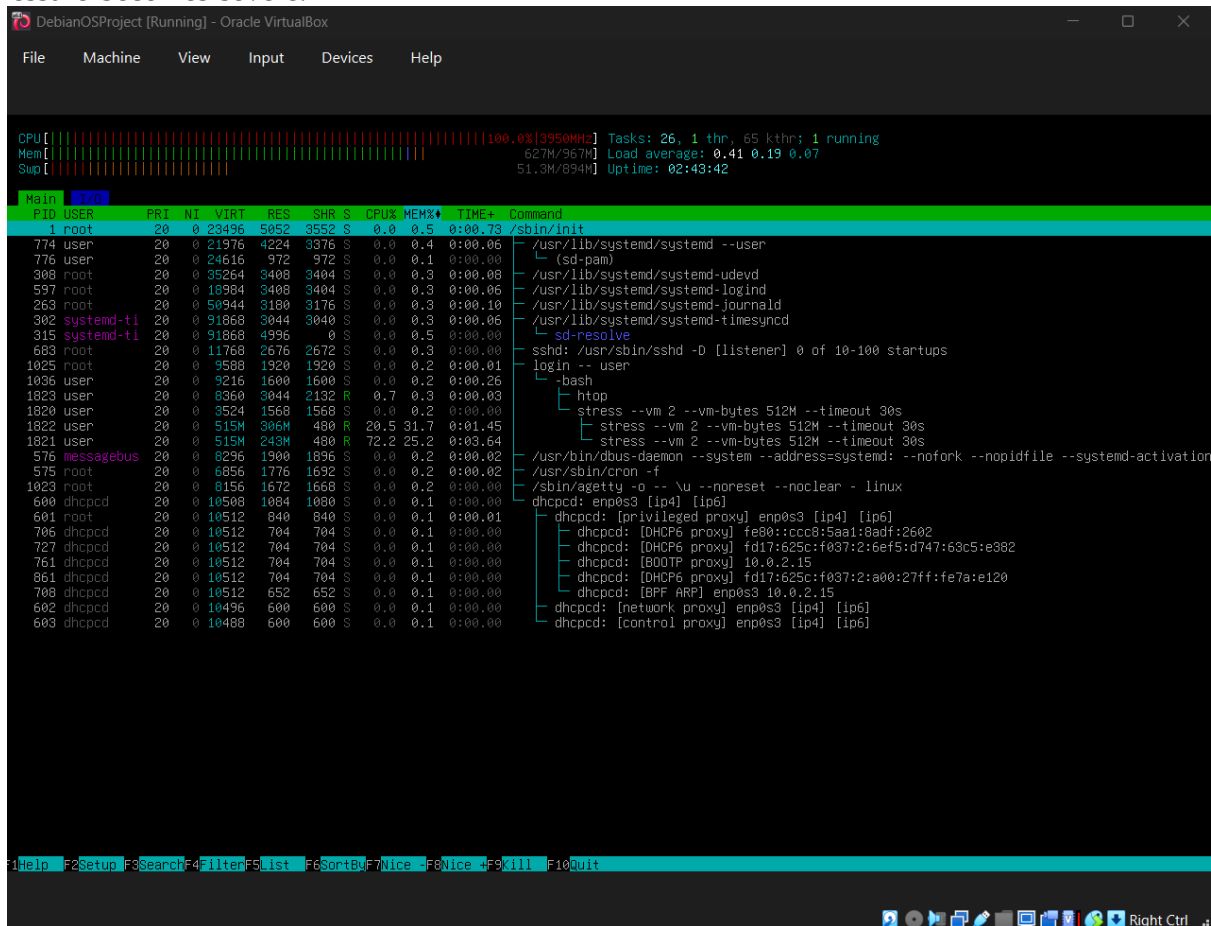
htop display during CPU stress test, showing all CPU cores at or near 100% utilization and elevated load average.

4.6.3 Memory Stress Test

A memory allocation stress test was performed to observe how the kernel's memory manager handles pressure:

```
stress --vm 2 --vm-bytes 512M --timeout 30s
```

This spawns worker processes that continuously allocate and free large blocks of memory, exercising the kernel's physical memory allocator and the swap subsystem if memory pressure becomes severe.



htop display during memory stress test, showing elevated memory bar usage and swap activity.

4.6.4 Combined Stress Test and Performance Summary

A combined stress scenario was run to simulate a realistic high-load condition:

```
stress-ng --cpu 1 --vm 1 --vm-bytes 256M --io 2 --timeout 60s --metrics-brief
```

The --metrics-brief flag causes stress-ng to output a performance summary upon completion, reporting the number of operations (bogo-ops) completed per second for each stressor. This provides a reproducible, quantitative benchmark of the virtual machine's performance.

```
DebianOSProject [Running] - Oracle VirtualBox
File Machine View Input Devices Help

CPU [|||||] 100.0% [3958MHz] Tasks: 29, 1 thr, 66 kthr: 1 running
Mem [|||||] 448M/367M Load average: 1.95 0.62 0.25
Sup [|||||] 18.6M/894M Uptime: 02:49:20

Main [|||||]
PID USER PRI NI VIRT RES SHR S CPU% MEM%+ TIME+ Command
1 root 20 0 23496 3148 2752 S 0.0 0.3 0:00.74 /sbin/init
263 root 20 0 50944 4532 4256 S 0.0 0.5 0:00.12 /usr/lib/systemd/systemd-journald
597 root 20 0 18984 1756 1636 S 0.0 0.2 0:00.07 /usr/lib/systemd/systemd-logind
302 systemd-t 20 0 91868 1716 1612 S 0.0 0.2 0:00.06 /usr/lib/systemd/systemd-timesyncd
315 systemd-t 20 0 91868 1716 0 S 0.0 0.2 0:00.00 sd-resolve
308 root 20 0 35264 1292 1196 S 0.0 0.1 0:00.00 /usr/lib/systemd/systemd-udev
600 dhcpcd 20 0 10508 1252 972 S 0.0 0.1 0:00.00 dhcpcd: enp0s3 [ip4] [ip6]
601 root 20 0 10512 1004 788 S 0.0 0.1 0:00.01 dhcpcd: [privileged proxy] enp0s3 [ip4] [ip6]
706 dhcpcd 20 0 10512 264 260 S 0.0 0.0 0:00.00 dhcpcd: [DHCP6 proxy] fe80::ccc8:5aa1:8adf:2602
708 dhcpcd 20 0 10512 264 260 S 0.0 0.0 0:00.00 dhcpcd: [BPF ARP] enp0s3 10.0.2.15
727 dhcpcd 20 0 10512 264 260 S 0.0 0.0 0:00.00 dhcpcd: [DHCP6 proxy] fd17:625c:f037:2:6ef5:d747:63c5:e382
761 dhcpcd 20 0 10512 264 260 S 0.0 0.0 0:00.00 dhcpcd: [BOOTP proxy] 10.0.2.15
861 dhcpcd 20 0 10512 264 260 S 0.0 0.0 0:00.00 dhcpcd: [DHCP6 proxy] fd17:625c:f037:2:a00:27ff:fe7a:e120
602 dhcpcd 20 0 10496 556 392 S 0.0 0.1 0:00.00 dhcpcd: [network proxy] enp0s3 [ip4] [ip6]
603 dhcpcd 20 0 10488 408 360 S 0.0 0.0 0:00.00 dhcpcd: [control proxy] enp0s3 [ip4] [ip6]
575 root 20 0 6856 904 820 S 0.0 0.1 0:00.02 /usr/sbin/cron -f
1025 root 20 0 9580 326 332 S 0.0 0.0 0:00.01 login -- user
1056 user 20 0 9216 2672 1824 S 0.0 0.3 0:00.27 -bash
1840 user 20 0 84476 43908 42360 S 0.0 4.4 0:00.04 stress-ng --cpu 1 --vm 1 --vm-bytes 256M --io 2 --timeout 60s --metrics-brief
1841 user 20 0 84476 10184 5068 R 39.1 1.0 0:15.06 stress-ng-cpu [run]
1843 user 20 0 84476 3464 1916 D 5.3 0.3 0:02.23 stress-ng-io [run]
1844 user 20 0 84476 3404 1856 D 5.3 0.3 0:02.23 stress-ng-io [run]
1842 user 20 0 84480 3332 1780 S 0.0 0.3 0:00.00 stress-ng-vm [wait]
1846 user 20 0 370M 291M 2008 R 38.4 30.1 0:15.91 stress-ng-vm [run]
1847 user 20 0 8364 4708 3464 R 0.0 0.5 0:00.16 htop
576 messagebus 20 0 8296 332 324 S 0.0 0.0 0:00.02 /usr/bin/dbus-daemon --system --address=systemd: --nofork --nopidfile --systemd-activation
683 root 20 0 11768 332 324 S 0.0 0.0 0:00.00 sshd: /usr/sbin/ssh -O [listener] 0 of 10-100 startups
1023 root 20 0 8156 328 324 S 0.0 0.0 0:00.00 /sbin/agetty -o - \u --nolinux --noclear - linux
774 user 20 0 21976 320 316 S 0.0 0.0 0:00.06 /usr/lib/systemd/systemd --user
776 user 20 0 24616 252 248 S 0.0 0.0 0:00.00 (sd-pam)

Help F2Setup F3Search F4Filter F5List F6SortBy F7Nice F8Nice F9Kill F10Quit
```

htop during the combined stress test, showing simultaneous CPU, memory, and I/O load.

```
DebianOSProject [Running] - Oracle VirtualBox
File Machine View Input Devices Help

user@DebianOSProject:~$ stress-ng --cpu 1 --vm 1 --vm-bytes 256M --io 2 --timeout 60s --metrics-brief
stress-ng: info: [1858] setting to a 1 min run per stressor
stress-ng: info: [1858] dispatching hogs: 1 cpu, 1 vm, 2 io
stress-ng: info: [1861] io: this is a legacy I/O sync stressor, consider using iomix instead
stress-ng: info: [1860] vm: using 256M per stressor instance (total 256M of 693.81M available memory)
stress-ng: metric: [1851] stressor bogo ops real time usr time sys time bogo ops/s bogo ops/s
stress-ng: metric: [1851] (secs) (secs) (secs) (real time) (usr+sys time)
stress-ng: metric: [1851] cpu 12597 60.00 11.70 0.31 209.94 1048.96
stress-ng: metric: [1851] vm 1217779 60.07 10.97 1.05 20272.80 101257.62
stress-ng: metric: [1851] io 105914 59.99 0.56 4.71 1765.50 20077.60
stress-ng: info: [1851] skipped: 0
stress-ng: info: [1851] passed: 4: cpu (1) vm (1) io (2)
stress-ng: info: [1851] failed: 0
stress-ng: info: [1851] metrics untrustworthy: 0
stress-ng: info: [1851] successful run completed in 1 min
stress-ng: metric: [1858] stressor bogo ops real time usr time sys time bogo ops/s bogo ops/s
stress-ng: metric: [1858] (secs) (secs) (secs) (real time) (usr+sys time)
stress-ng: metric: [1858] cpu 12262 60.00 11.72 0.28 204.36 1022.16
stress-ng: metric: [1858] vm 1185280 60.04 10.81 1.22 19743.03 98528.08
stress-ng: metric: [1858] io 104591 60.00 0.50 4.86 1743.24 19524.32
stress-ng: info: [1858] skipped: 0
stress-ng: info: [1858] passed: 4: cpu (1) vm (1) io (2)
stress-ng: info: [1858] failed: 0
stress-ng: info: [1858] metrics untrustworthy: 0
stress-ng: info: [1858] successful run completed in 1 min
[1]+ Done stress-ng --cpu 1 --vm 1 --vm-bytes 256M --io 2 --timeout 60s --metrics-brief
user@DebianOSProject:~$
```

stress-ng --metrics-brief summary output showing bogo-ops/s for each stressor, serving as a performance baseline for the Debian VM.

4.6.5 Observations and Analysis

The stress tests produced several observable findings consistent with the kernel behaviour described in Part One. During the CPU stress test, htop showed the virtual CPU core reaching approximately 100% utilization. If the system was allocated more than one CPU core, the stress

worker processes would have been distributed evenly across all cores. This reflects the behaviour of the Completely Fair Scheduler (CFS): because all worker processes were launched simultaneously with equal priority (nice value is 0), CFS assigned each an identical weight and ensured that no single worker accumulated disproportionate CPU time. The PRI and NI columns in htop confirmed uniform priority across all workers. During the memory stress test, the memory bar in htop rose to approximately 627 MB of 967 MB, with swap activity. This behaviour is consistent with the demand paging and LRU page replacement mechanisms described in Section 2.4.12: the kernel allocated physical frames on demand as the stress workers wrote to memory, and pages were evicted to swap once the inactive list was exhausted. The combined stress-ng run produced a bogo-ops/s rate of around 1,030 for the CPU stressor and 100,000 for the VM stressor, providing a quantitative performance baseline for this Debian virtual machine configuration. Throughout all tests, the system remained responsive, indicating that the kernel's scheduler did not allow the stress workers to fully starve other processes such as htop itself, and that is a consequence of CFS's pre-emptive, time-sliced design.

4.7 Summary

Part Three of this case study provided a practical, hands-on implementation of the theoretical concepts discussed in Parts One and Two. Using a Debian GNU/Linux virtual machine hosted on VirtualBox, the system was configured, installed, and equipped with monitoring and tracing tools to enable direct observation of operating system behaviour at runtime.

The process and thread analysis conducted in Section 4.4 demonstrated in practice what Part One described theoretically: that Linux represents both processes and threads through the same `task_struct` kernel structure, and that parent-child relationships established via `fork()` are directly visible through tools like htop.

System call tracing in Section 4.5 reinforced the memory management and process lifecycle discussions from Part One. The `clone()` calls captured by strace illustrated how a single kernel primitive underpins both process and thread creation, while file operation traces confirmed the role of the kernel as the mandatory intermediary for all hardware access, which is a point central to the security model discussed in Part Two

Finally, the stress testing performed in Section 4.6 provided an overview of the kernel's physical memory allocator and scheduler under load, complementing the theoretical treatment of the buddy system, SLUB allocator, and demand paging in Sections 2.3.5 through 2.3.12.

5. Conclusion

This case study has provided a detailed examination of Debian GNU/Linux 13.4 "Trixie" from multiple perspectives — technical, ethical, and practical — demonstrating that it represents a mature and well-engineered operating system grounded in sound design principles.

From a process and thread management standpoint, Debian's adoption of the EEVDF scheduler marks a meaningful evolution beyond the long-standing CFS model, delivering improved fairness and latency characteristics, particularly for mixed workloads on multi-core systems. The unified task_struct model and NPTL threading implementation provide a robust, POSIX-compliant foundation that scales from embedded hardware to enterprise servers. The introduction of the sched_ext framework further underscores Debian's forward-looking approach, enabling eBPF-based scheduler customization without kernel recompilation.

The synchronization analysis revealed a comprehensive and carefully layered set of primitives — from fast-path futex-based mutexes and POSIX semaphores to spinlocks, read-write locks, and the lock-free RCU mechanism — all underpinned by the kernel's lockdep tool, which enforces lock ordering conventions to prevent deadlocks at development time. These mechanisms collectively satisfy the classical requirements of mutual exclusion, progress, and bounded waiting described in OS theory.

Memory management in Debian 13 demonstrated the practical effectiveness of combining the Buddy System allocator for physical memory, the SLUB allocator for kernel objects, demand paging with Copy-on-Write, and LRU-based page reclamation. The /proc filesystem provided direct visibility into each of these subsystems, confirming that theory and runtime behavior are closely aligned.

The kernel architecture evaluation found that Debian's monolithic-yet-modular design strikes an effective balance between execution efficiency and operational flexibility. The clean separation between kernel mode and user mode, enforced through the system-call interface, remains one of the most important structural features of the Linux kernel, providing both performance and protection. Loadable kernel modules extend this architecture without compromising its fundamental integrity.

Debian's security model proved to be layered and architecturally embedded, relying not on any single mechanism but on the combined effect of privilege separation, user and group-based access control, pluggable authentication, and kernel-level security enforcement through Linux Security Modules. The ethical analysis further showed that Debian's commitment to the GPL, minimal data collection, transparent governance, and responsible default configurations reflects an operating system designed with genuine respect for its users.

The hands-on implementation validated these theoretical observations directly. Stress testing confirmed the scheduler's ability to maintain system responsiveness under full CPU and memory load, strace traces illustrated the kernel's role as the mandatory intermediary for all privileged operations, and htop demonstrated the process hierarchy and resource usage patterns that the earlier sections described analytically.

In summary, Debian 13.4 "Trixie" is a technically sophisticated, ethically principled, and practically effective operating system. Its design reflects decades of accumulated engineering experience in the Linux kernel and Debian project communities. For students and practitioners of operating systems, it offers an exceptionally transparent and well-documented environment in which core OS concepts — scheduling, synchronization, memory management, kernel architecture, and security — can be studied both theoretically and in direct practice.

6. References

Section 2.1 References

arXiv. (2026a). Towards agentic OS: An LLM agent framework for Linux schedulers.

<https://arxiv.org/html/2509.01245v3>

arXiv. (2026b). Towards agentic OS: An LLM agent framework for Linux schedulers.

<https://arxiv.org/html/2509.01245v2>

Bovet, D. P., & Cesati, M. (2005). Understanding the Linux kernel: From I/O ports to process management. O'Reilly Media.

https://books.google.com/books/about/Understanding_the_Linux_Kernel.html?id=h0lItXyJ8aIC

Debian Handbook. (2026). About the book: The Debian administrator's handbook.

<https://debian-handbook.info/about-the-book/>

Debian Manpages. (2026a). nptl(7) — manpages — Debian testing.

<https://manpages.debian.org/testing/manpages/nptl.7.en.html>

Debian Manpages. (2026b). systemd.resource-control(5).

<https://manpages.debian.org/stretch/systemd/systemd.resource-control.5.en.html>

Debian Tracker. (2026). Accepted glibc 2.43-1 (source) into experimental.

<https://tracker.debian.org/news/1711030/accepted-glibc-243-1-source-into-experimental/>

Debian Wiki. (2026). DebianReleases. <https://wiki.debian.org/DebianReleases>

Debian. (2026a). Debian — Wikipedia. <https://en.wikipedia.org/wiki/Debian>

Debian. (2026b). What's new in Debian 13: Release notes documentation.

<https://www.debian.org/releases/stable/release-notes/whats-new.html>

Debian. (2026c). Updated Debian 13: 13.4 released.

<https://www.debian.org/News/2026/20260314>

Debian. (2026d). [SECURITY] [DLA 4476-1] linux-6.1 security update.

<https://www.debian.org/lts/security/dla-4476-1>

Debian. (2026e). Issues to be aware of for trixie: Release notes documentation.

<https://www.debian.org/releases/trixie/release-notes/issues.html>

dohost.us. (2026). Exploring cgroups v1 and cgroups v2: Understanding the evolution of resource control. <https://dohost.us/index.php/2025/10/11/exploring-cgroups-v1-and-cgroups-v2-understanding-the-evolution-of-resource-control/>

ETH Zurich. (2026). Resource control. ISG D-PHYS Readme Wiki.

https://readme.phys.ethz.ch/linux/workstation/resource_control/

freedesktop.org. (2026). systemd.resource-control.

<https://www.freedesktop.org/software/systemd/man/systemd.resource-control.html>

GeeksforGeeks. (2026). top command in Linux with examples.

<https://www.geeksforgeeks.org/linux-unix/top-command-in-linux-with-examples/>

Hasegawa, T., Aoyama, K., & Suguri, H. (2021). Thread-aware mechanism to enhance inter-node load balancing for multithreaded applications on NUMA systems. Applied Sciences, 11(14), 6486. <https://www.mdpi.com/2076-3417/11/14/6486>

Hill, S. (2006). Native POSIX Threads Library (NPTL) support for uClibc. Proceedings of the Linux Symposium, 1, 409–420. <https://www.landley.net/kdocs/ols/2006/ols2006v1-pages-409-420.pdf>

HostMyCode. (2026). 7 critical Linux kernel security patches every VPS administrator must apply in 2026: CVE analysis and automated patching.

<https://www.hostmycode.com/tutorials/7-critical-linux-kernel-security-patches-vps-administrator-2026-cve-analysis-automated-patching>

Hu, H., & Li, H. (2026). Linux kernel programming — Lecture 07: CPU scheduling: CFS to EEVDF [Lecture slides]. Virginia Tech. <https://people.cs.vt.edu/huaicheng/lkp-sp26/slides/L07-sched3.pdf>

Klipper. (2026). KB: Klipper systemd service. CPU Nice/Weight & etc [Forum post].

<https://klipper.discourse.group/t/kb-klipper-systemd-service-cpu-nice-weight-etc/22757>

Kubernetes. (2026). About cgroup v2. <https://kubernetes.io/docs/concepts/architecture/cgroups/>

Lefeuvre, H., Bhatotia, P., Huici, F., Handurukande, S., Rotsos, C., & Olivier, P. (2021). FlexOS: Making OS isolation flexible. Proceedings of HotOS '21.

https://pure.manchester.ac.uk/ws/portalfiles/portal/195993673/flexos_hotos21.pdf

Linux Kernel Docs. (2026a). EEVDF scheduler. The Linux Kernel Documentation.

<https://docs.kernel.org/scheduler/sched-eevdf.html>

Linux Kernel Docs. (2026b). CFS scheduler. The Linux Kernel Documentation.

<https://docs.kernel.org/scheduler/sched-design-CFS.html>

Linux Kernel Docs. (2026c). Extensible scheduler class. The Linux Kernel Documentation.

<https://docs.kernel.org/scheduler/sched-ext.html>

LinuxBlog.io. (2026). htop and top alternatives: Glances, nmon. <https://linuxblog.io/alternatives-top-htop/>

LWN.net. (2026). Completing the EEVDF scheduler. <https://lwn.net/Articles/969062/>

man7.org. (2026). nptl(7) — Linux manual page. <https://man7.org/linux/man-pages/man7/nptl.7.html>

MangoHost. (2026). atop tutorial: Advanced performance monitoring for Linux servers.

<https://mangohost.net/blog/atop-tutorial-advanced-performance-monitoring-for-linux-servers/>

MasterDC. (2026). Debian 13 Trixie: Key updates and recommendations for users.

<https://www.masterdc.com/help/debian-13-trixie-key-updates-recommendations-for-users/>

Mintlify. (2026). Process scheduler — Linux kernel.

<https://www.mintlify.com/torvalds/linux/internals/process-scheduler>

Never Ending Security. (2015). 6 quick tools to monitor system resources on Linux.

<https://neverendingsecurity.wordpress.com/2015/03/14/6-quick-tools-to-monitor-system-resources-on-linux/>

Phoronix. (2026). Linux 7.0 released with new hardware support, optimizations & self-healing

XFS. <https://www.phoronix.com/news/Linux-7.0-Released>

Reddit. (2026). Switching from CFS scheduling to EEVDF scheduling in an embedded Linux system [Forum post].

https://www.reddit.com/r/embedded/comments/1skqsap/switching_from_cfs_scheduling_to_eevdf_scheduling/

sched-ext.com. (2026). Overview. <https://sched-ext.com/docs/OVERVIEW>

Semantic Scholar. (2026). Native POSIX Threads Library (NPTL) support for uClibc [Paper summary]. [https://www.semanticscholar.org/paper/Native-POSIX-Threads-Library-](https://www.semanticscholar.org/paper/Native-POSIX-Threads-Library-(NPTL)-Support-for-Hill/0cd8c9aa136b49de3d9c1fa08964eaf03bcf5b11)

[\(NPTL\)-Support-for-Hill/0cd8c9aa136b49de3d9c1fa08964eaf03bcf5b11](https://www.semanticscholar.org/paper/Native-POSIX-Threads-Library-(NPTL)-Support-for-Hill/0cd8c9aa136b49de3d9c1fa08964eaf03bcf5b11)

Sematext. (2026). 15 best Ubuntu monitoring tools (free & paid) in 2025.

<https://sematext.com/blog/ubuntu-monitoring-tools/>

SentinelOne. (2026a). CVE-2025-21700: Linux kernel privilege escalation flaw.

<https://www.sentinelone.com/vulnerability-database/cve-2025-21700/>

SentinelOne. (2026b). CVE-2025-21647: Linux kernel buffer overflow vulnerability.

<https://www.sentinelone.com/vulnerability-database/cve-2025-21647/>

systemd.io. (2026). New control group interfaces.

https://systemd.io/CONTROL_GROUP_INTERFACE/

Ulrich, D., Bhattiprolu, S., & Bhargava, V. (2006). Linux threading models compared:

LinuxThreads and NPTL. Universidad Nacional del Sur.

<http://cs.uns.edu.ar/~jechaiz/sosd/clases/extras/03-LinuxThreads%20and%20NPTL.pdf>

Wikipedia. (2026a). Cgroups. <https://en.wikipedia.org/wiki/Cgroups>

Wikipedia. (2026b). Linux kernel version history.

https://en.wikipedia.org/wiki/Linux_kernel_version_history

Wikipedia. (2026c). Earliest eligible virtual deadline first scheduling.

https://en.wikipedia.org/wiki/Earliest_eligible_virtual_deadline_first_scheduling

Zaccaria, V. (2026). Introductory notes on the new Linux scheduler (EEVDF).

<https://www.vittoriozaccaria.net/blog/notes-on-linux-eevdf>

ZX2C4 Git Repository. (2026). Start documenting the EEVDF scheduler.

<https://git.zx2c4.com/linux->

[rng/commit/?id=602bce7e5edeb0d701ee2074fe512c4d793dac0d](https://git.zx2c4.com/linux-rng/commit/?id=602bce7e5edeb0d701ee2074fe512c4d793dac0d)

Sections 2.2 and 2.3 References

Debian Project. (2025). Debian 13 "Trixie" Release Announcement.

<https://www.debian.org/News/2025/20250809>

Debian Project. (2026). Release Notes for Debian 13 (Trixie).

<https://www.debian.org/releases/trixie/release-notes.en.pdf>

GeeksforGeeks. (2025). Allocating Kernel Memory (Buddy System and Slab System).

<https://www.geeksforgeeks.org/operating-systems/operating-system-allocating-kernel-memory-buddy-system-slab-system/>

Linux Journal. (n.d.). Linux Memory Management: Understanding Page Tables, Swapping, and Memory Allocation. <https://www.linuxjournal.com/content/linux-memory-management-understanding-page-tables-swapping-and-memory-allocation>

Linux Kernel Labs. (n.d.). Memory Management. <https://linux-kernel-labs.github.io/refs/heads/master/lectures/memory-management.html>

MasterDC. (2025). Debian 13 Trixie: Key Updates and Recommendations for Users.

<https://www.masterdc.com/help/debian-13-trixie-key-updates-recommendations-for-users/>

Oracle Linux Blog. (2022). Linux SLUB Allocator Internals and Debugging, Part 1.

<https://blogs.oracle.com/linux/linux-slub-allocator-internals-and-debugging-1>

PUDN Club. (2025). Linux Kernel Mutex Explained. <https://www.pudn.club/programming/linux-kernel-mutex-explained-synchronization-and-internals/>

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating System Concepts (10th ed.). Wiley.

[Chapters 6, 7, 8, 9, 10]. <https://os.ecci.ucr.ac.cr/slides/Abraham-Silberschatz-Operating-System-Concepts-10th-2018.pdf>

sysplay.in. (2015). Concurrency Management in Linux Kernel. <https://sysplay.in/blog/linux-kernel-internals/2015/06/concurrency-management-in-linux-kernel/>

syst3mfailure.io. (2025). A Quick Dive Into The Linux Kernel Page Allocator. <https://syst3mfailure.io/linux-page-allocator/>

Wikipedia. (2025). Slab allocation. https://en.wikipedia.org/wiki/Slab_allocation

Sections 2.4 and 2.5 References

Debian Project. (n.d.). Debian documentation. Retrieved April 15, 2026, from

<https://www.debian.org/doc/>

Linux Kernel Organization. (n.d.). Linux kernel documentation. Retrieved April 15, 2026, from

<https://docs.kernel.org/>

Linux Kernel Organization. (n.d.). Linux Security Module (LSM) framework. Retrieved April

15, 2026, from <https://docs.kernel.org/security/lsm.html>

Love, R. (2010). Linux kernel development (3rd ed.). Pearson Education.

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating system concepts (10th ed.).

Wiley.

Ethical Considerations

DebianReleases - Debian Wiki. (n.d.).

<https://wiki.debian.org/DebianReleases#:~:text=Security%20updates%20are%20provided%20by,LTS%20and%20LTS%2FExtended%20projects.>

Debian - Reasons to use Debian. (n.d.).

https://www.debian.org/intro/why_debian#:~:text=Debian%20is%20Free%20Software.,an%20automated%20network%20installation%20tool.

Debian - Privacy policy. (n.d.).

[https://www.debian.org/legal/privacy#:~:text=The%20Debian%20Project%20is%20a,hid e%20our%20problems%20\(%233\).](https://www.debian.org/legal/privacy#:~:text=The%20Debian%20Project%20is%20a,hid e%20our%20problems%20(%233).)

Debian Constitution. (2022). Debian.org. <https://www.debian.org/devel/constitution>

Debian Social Contract. (2022). Debian.org. https://www.debian.org/social_contract

Debian - Security Information. (2026). Debian.org. <https://www.debian.org/security/>

Debian - What does free mean? (2024). Debian.org. <https://www.debian.org/intro/free>

sudo - Debian Wiki. (2026). Debian.org. <https://wiki.debian.org/sudo>

Nestor, M. (2026, March 14). *Debian 13.4 “Trixie” Released with 111 Bug Fixes and 67 Security Updates*. 9to5Linux. <https://9to5linux.com/debian-13-4-trixie-released-with-111-bug-fixes-and-67-security-updates#:~:text=67%20Security%20Updates-.Debian%2013.4%20%E2%80%9CTrixie%E2%80%9D%20Released%20with%20111%20Bug%20Fixes%20and%2067,with%20the%20previous%20ISO%20releases.>

Appendix A: System Information

Debian Version: Debian 13.4 (Trixie)

Kernel Version: 6.12.74-amd64

Architecture: x86_64

VirtualBox Version: 7.0.12

Host OS: [Host Operating System]

Appendix B: Key Commands Reference

Command	Purpose
top	Real-time process monitoring
htop	Interactive process viewer with tree view
ps aux	Snapshot of all running processes
strace -p PID	Attach strace to a running process
strace -e trace=file cmd	Trace file-related system calls
free -m	Display memory usage in megabytes
vmstat 1	Virtual memory statistics every 1 second
stress --cpu N	Generate CPU load with N workers
cat /proc/meminfo	Detailed memory information
lsmod	List loaded kernel modules